

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

AI-Powered Smart Aquaculture Assistant – Smart Fish Stress Prediction & Water Quality Management System

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

Dharshish.K.G (23619112)

Haamidsha.D (23619113)

Mohamed Sadhath.J (23619122)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "LLM Probe: Activation-Based Detection of Memorization and Hallucination in Large Language Models" is the bonafide work of **Dharshish.K.G (23619112), Haamidsha.D (23619113), Mohamed Sadhath.J (23619122)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR

Mrs. F. Benasir Begam, M.E., (Ph.D.)
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI

K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya., for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, Mrs. F. Benasir Begam, who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator Ms. S.J.Kavitha. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

DHARSHISH.K.G **23619112**

HAAMIDSHA.D **23619113**

MOHAMED SADHATH.J **23619122**

TABLE OF CONTENTS

SECTION	PAGE
ABSTRACT	6
LIST OF FIGURES	7
LIST OF TABLES	8
LIST OF ABBREVIATIONS	9
CHAPTER 1 – INTRODUCTION	10
1.1 General	10
1.2 Problem Description	10
1.3 Existing System	11
1.4 Proposed System	11
1.5 Objectives	12
1.6 Scope of the Project	12
1.7 Contributions	12
1.8 Modules	12
1.9 Summary	12
CHAPTER 2 – LITERATURE REVIEW	13
2.1 Aquaculture Water Quality Monitoring	13
2.2 IoT in Aquaculture	13
2.3 Machine Learning for Fish Health	14
2.4 Smart Water Quality Systems	14
2.5 Existing Systems and Limitations	14
2.6 Research Gap	14
2.7 Summary	14
CHAPTER 3 – SYSTEM DESIGN	15
3.1 Existing System	15
3.2 Proposed System	15
3.3 System Architecture	15
3.4 Overall Workflow	16
3.5 System Modules	16
3.6 Data Flow	17
3.7 Hardware Requirements	17
3.8 Software Requirements	18
3.9 Summary	18
CHAPTER 4 – IMPLEMENTATION AND METHODOLOGY	19
4.1 Dataset Preparation	19
4.2 Data Preprocessing	19
4.3 Random Forest Model	20
4.4 Model Training and Evaluation	20
4.5 ESP32 Sensor Implementation	20
4.6 Streamlit Dashboard	21
4.7 Rule-Based Live Health Logic	21
4.8 Calibration and Limitations	22
4.9 GitHub and Reproducibility	22
4.10 Summary	22

CHAPTER 5 – RESULTS AND DISCUSSION	23
5.1 Dataset Results	23
5.2 Model Results	24
5.3 Sensor Results	24
5.4 Dashboard Results	25
5.5 Functional Testing	26
5.6 Discussion	27
5.7 Comparison with Existing Approaches	27
5.8 Summary	27
CHAPTER 6 – LIVE SYSTEM DEMONSTRATION	38
6.1 Overview	38
6.2 ESP32 Live Monitoring	38
6.3 Streamlit Live Dashboard	39
6.4 Sample Stable Case	39
6.5 Sample At-Risk Case	40
6.6 Sample High-Risk Case	40
6.7 Summary	41
CHAPTER 7 – CONCLUSION AND FUTURE WORK	45
7.1 Conclusion	45
7.2 Limitations	45
7.3 Future Scope	46
7.4 Summary	46
CHAPTER 8 – REFERENCES	47
CHAPTER 9 – APPENDIX	49
CHAPTER 10 – SOURCE CODE	51
CHAPTER 11 – CONFERENCE APPLICATION	59

ABSTRACT

Aquaculture is an important source of food, employment and economic activity, but successful fish production depends strongly on suitable water-quality conditions. Fish can experience stress when environmental conditions such as temperature and pH move outside acceptable ranges. Changes may occur gradually or suddenly, and manual monitoring at selected intervals may fail to capture short-term variations. An integrated monitoring approach can therefore provide timely information for observation and management of aquaculture ponds.The Smart Aquaculture Assistant is a low-cost prototype combining IoT sensing, Python data processing, machine-learning experimentation and a Streamlit web dashboard. An ESP32 acts as the central controller of the live prototype. Temperature, Dissolved Oxygen, pH and Turbidity were selected as model features, with Health Status as the target. A Random Forest Classifier with 100 estimators and random_state equal to 42 was trained using a train-test split. The model achieved approximately 99.89% accuracy on the selected test split. This result demonstrates the feasibility of supervised classification on the historical dataset, but it is not presented as a guarantee of field performance.An important technical distinction is maintained between the offline machine-learning experiment and the live hardware prototype. The Random Forest model expects Temperature, Dissolved Oxygen, pH and Turbidity, whereas the live prototype currently provides Temperature, pH, TDS voltage and water-sensor readings. Therefore, the live values are not incorrectly passed to the historical model. The current dashboard instead uses transparent rule-based condition assessment and produces three overall outcomes: Stable, At Risk and High Risk. This separation makes the project results technically clear and reproducible.The project documents sensor wiring, ESP32 programming, serial communication, Python processing, dashboard development, machine-learning preparation, testing and result interpretation. Prototype testing showed realistic DS18B20 temperature readings, TDS voltage readings and changing HW-038 values, while the pH interface required additional calibration and electrical troubleshooting. The main contribution is the integration of affordable sensing, reproducible software and an easy-to-understand monitoring interface. Future work includes calibrated dissolved-oxygen and turbidity sensors, improved pH and TDS calibration, synchronized live data collection, retraining the model with live-compatible variables, cloud connectivity and field validation.

Keywords — Smart aquaculture, IoT, ESP32, water quality, fish health, Random Forest, machine learning, Streamlit, pH, TDS.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture of Smart Aquaculture Assistant	16
Figure 3.2	Overall Data Flow	17
Figure 4.1	Dataset-to-Model Development Workflow	24
Figure 4.2	ESP32 Connection Diagram	24
Figure 4.3	Streamlit Dashboard Architecture	24
Figure 5.1	Health Status Distribution in Dataset	33
Figure 5.2	Random Forest Model Evaluation	34
Figure 5.3	ESP32 Hardware Prototype	34
Figure 5.4	GitHub Repository Structure	34
Figure 6.1	Live ESP32 Sensor Output	35
Figure 6.2	Stable Dashboard Output	45
Figure 6.3	At-Risk Dashboard Output	45
Figure 6.4	High-Risk Dashboard Output	46
Figure 6.5	Final Working Prototype	46

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Project Objectives	11
Table 3.1	Major System Modules	17
Table 3.2	Hardware Requirements	17
Table 3.3	Software Requirements	18
Table 4.1	Dataset Information	23
Table 4.2	Machine-Learning Features	24
Table 4.3	Live Sensor Parameters	24
Table 5.1	Health Status Distribution	33
Table 5.2	Model Evaluation Result	33
Table 5.3	Sample Sensor Test Results	34
Table 5.4	Functional Testing Summary	35
Table 5.5	Comparison with Existing Approaches	35
Table 6.1	Live Demonstration Test Cases	39
Table A.1	Project Directory Structure	54
Table A.2	Important Commands	54

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
IoT	Internet of Things
RF	Random Forest
ESP32	Espressif ESP32 Microcontroller
TDS	Total Dissolved Solids
DO	Dissolved Oxygen
pH	Potential of Hydrogen
NTU	Nephelometric Turbidity Unit
CSV	Comma-Separated Values
IDE	Integrated Development Environment
USB	Universal Serial Bus
API	Application Programming Interface
UI	User Interface
GUI	Graphical User Interface
CPU	Central Processing Unit
RAM	Random Access Memory
COM	Communication Port
PWM	Pulse Width Modulation
CSV/XLSX	Dataset file formats
Git	Version Control System

CHAPTER 1
INTRODUCTION

1.1 General

Aquaculture is the controlled farming of aquatic organisms such as fish, shrimp and other aquatic species. The productivity of an aquaculture pond depends not only on feed and stocking density but also on the physical and chemical condition of the water. Parameters such as temperature, pH, dissolved oxygen, turbidity and dissolved substances can change with weather, feeding, biological activity and water exchange.In conventional pond management, water quality is often checked manually at selected times. Manual checking can miss rapid changes between measurements. A smart monitoring system can continuously collect sensor data and present it in a simple dashboard so that abnormal conditions can be noticed earlier.The Smart Aquaculture Assistant combines an ESP32-based sensing layer, Python software, a Streamlit dashboard and a machine-learning study. The main aim is not to replace an aquaculture expert, but to provide a technology-assisted monitoring and early-warning tool.

1.2 Problem Description

Fish experience stress when the surrounding water moves outside suitable environmental conditions. If such changes are not detected early, fish may reduce feeding, grow slowly, become more susceptible to disease or suffer mortality. Small-scale farmers and student project users may not have access to continuous automated monitoring.The main problem addressed by this project is the lack of a simple integrated prototype that combines water sensing, live visualization, fish-health status indication and machine-learning based analysis in one system. The system must also be affordable and understandable for educational and prototype use.

1.3 Existing System

Traditional aquaculture monitoring commonly depends on manual observation and periodic testing. Commercial monitoring solutions can provide automated measurements, but they may be costly and may require proprietary hardware, cloud subscriptions or specialized configuration. Some research systems focus only on a single parameter, while others monitor multiple parameters without connecting them to a simple fish-health decision layer.Another limitation is that water-quality monitoring and fish-health prediction are often treated as separate tasks. A farmer may receive a sensor reading but still need to decide whether the combined condition is safe for fish.

1.4 Proposed System

The proposed system uses an ESP32 as the central sensing controller. The prototype reads temperature, pH, TDS voltage and water-presence/level information. The readings are sent through

USB serial communication to a Streamlit application. The dashboard displays live values, individual sensor status and an overall fish-health status.The machine-learning component is developed separately using an aquaculture dataset. A Random Forest Classifier learns the relationship between Temperature, Dissolved Oxygen, pH and Turbidity and the Health Status label. This model is used as the project's offline ML study; the live prototype uses a rule-based decision layer because the current live sensor inputs do not match the model's training features.

1.5 Objectives

No.	Objective
1	Collect live aquaculture-related measurements using ESP32 and sensors.
2	Display sensor readings in a simple Streamlit dashboard.
3	Classify sensor conditions into Normal, At Risk or High Risk.
4	Provide an overall Stable, At Risk or High Risk fish-health status.
5	Train a Random Forest model using historical aquaculture data.
6	Evaluate the model and document its test-split performance.
7	Organize source code, dataset, model and documentation in GitHub.
8	Create a foundation for future calibrated ML and IoT automation.

Table 1.1: Project Objectives

1.6 Scope of the Project

The scope includes hardware sensing, serial communication, dashboard development, historical dataset analysis, machine-learning model training, model evaluation, documentation and prototype testing. The current project does not claim production-level deployment, calibrated TDS ppm measurement, automatic pond control or a field-validated live ML model.

1.7 Contributions

- Developed an integrated ESP32-based sensor monitoring prototype.
- Developed a Streamlit dashboard for live sensor visualization.
- Created sensor-level decision rules and an overall fish-health status.
- Prepared and trained a Random Forest classifier on historical aquaculture data.
- Documented hardware, software, ML and dashboard workflows.
- Organized the project for GitHub source-code submission.

1.8 Modules

- Sensor Acquisition Module – reads temperature, pH, TDS and water-sensor values.
- ESP32 Communication Module – formats readings and sends them through USB serial.

- Dashboard Module – receives serial data and displays live values.
- Health Assessment Module – converts sensor values into status categories.
- Machine Learning Module – trains and evaluates the historical-data Random Forest model.
- Documentation and Repository Module – stores code, model, dataset, images and project documentation.

1.10 Requirements and Traceability

1.10.1 Requirement Traceability Matrix

ID	Requirement	Implementation Evidence
FR-01	Read live temperature	DS18B20 on GPIO4 and Arduino temperature output
FR-02	Read water condition	HW-038 on GPIO32 with raw analog value
FR-03	Read TDS indicator	TDS interface on GPIO34; voltage calculation
FR-04	Read pH input	pH interface on GPIO35; prototype conversion
FR-05	Show live dashboard	Streamlit app.py with serial parser
FR-06	Generate overall status	Stable / At Risk / High Risk rule logic
FR-07	Support ML experiment	Jupyter notebook and saved Random Forest model
NFR-01	Maintain reproducibility	requirements.txt, GitHub structure and fixed serial labels

Traceability ensures that every major requirement can be linked to a hardware connection, software component or documented experiment. This is useful during review because the evaluator can ask for evidence of any claimed feature.

1.9 Summary

This chapter introduced the aquaculture problem, existing limitations, proposed solution, objectives, scope and modules. The next chapter reviews related work in IoT-based aquaculture monitoring and machine-learning based fish-health analysis.

CHAPTER 2
LITERATURE REVIEW

2.1 Aquaculture Water Quality Monitoring

Water quality is a major factor in fish production. Temperature influences fish metabolism and feeding behaviour. pH affects biological processes and the toxicity of some dissolved compounds. Dissolved oxygen is particularly important because fish depend on oxygen in water for respiration. Turbidity can indicate suspended material and changes in pond conditions.Continuous monitoring can provide more information than occasional manual measurements because it creates a sequence of observations. However, sensor readings need calibration, correct placement and maintenance to remain useful.

2.2 IoT in Aquaculture

The Internet of Things enables physical sensors to be connected to a microcontroller and communication system. In aquaculture, an IoT node can collect environmental measurements and transmit them to a local or cloud dashboard. ESP32-class controllers are attractive for educational prototypes because they provide analog and digital interfaces, wireless capability and low-cost development.The present prototype uses USB serial communication between the ESP32 and the computer. Wi-Fi/cloud communication is considered a future extension rather than a completed feature.

2.3 Machine Learning for Fish Health

Machine learning can learn patterns from historical water-quality and fish-health observations. Classification algorithms such as Decision Trees, Random Forest, Support Vector Machines and neural networks can be used when a labelled health category is available. The quality of the training data, feature selection, class balance and evaluation strategy directly influence the reliability of the resulting model.Random Forest is suitable for a student prototype because it combines multiple decision trees and can model non-linear relationships without requiring complex feature engineering. It also works well with tabular data and is straightforward to train using scikit-learn.

2.4 Smart Water Quality Systems

Many smart monitoring systems combine multiple sensors with a microcontroller and a user interface. Typical architectures include sensors, embedded processing, communication, a database or cloud service, analytics and a dashboard. Alert systems may use thresholds to notify users when a parameter becomes unsafe.A useful design principle is to separate raw measurement, processing and decision-making. This makes it easier to recalibrate a sensor without rewriting the complete dashboard.

2.5 Existing Systems and Limitations

Approach	Strength	Limitation
Manual testing	Simple and familiar	Not continuous; delayed detection
Single-parameter monitoring	Low complexity	Does not show combined water condition
Commercial monitoring	Integrated hardware and software	Can be expensive or proprietary
Cloud IoT system	Remote access and historical data	Needs network/cloud infrastructure
ML-only prediction	Can learn complex patterns	Requires suitable labelled data
Proposed prototype	Combines sensing, dashboard and health status	Needs calibration and field validation

Table 2.1: Comparison of Monitoring Approaches

2.6 Research Gap

The project addresses the practical gap between raw sensor readings and an understandable fish-health decision. Instead of showing only numerical measurements, the dashboard translates the current sensor values into status categories and an overall health indication. It also demonstrates how historical aquaculture data can be used to build an independent ML classifier.

2.7 Summary

This chapter reviewed the importance of water quality, IoT sensing, machine learning and dashboard-based monitoring. The review motivates an integrated prototype that combines embedded sensing with data analysis and simple decision support.

CHAPTER 3
SYSTEM DESIGN

3.1 Existing System

In the existing manual workflow, a user measures water parameters separately and interprets the results manually. This creates several opportunities for delay and human error. The user may also need different tools for data collection, prediction and visualization.

3.2 Proposed System

The proposed Smart Aquaculture Assistant contains a sensing layer, ESP32 controller, communication layer, processing and decision layer, and dashboard. The architecture is designed so that sensor readings are collected in a consistent text format and parsed by the Python application.

3.3 System Architecture

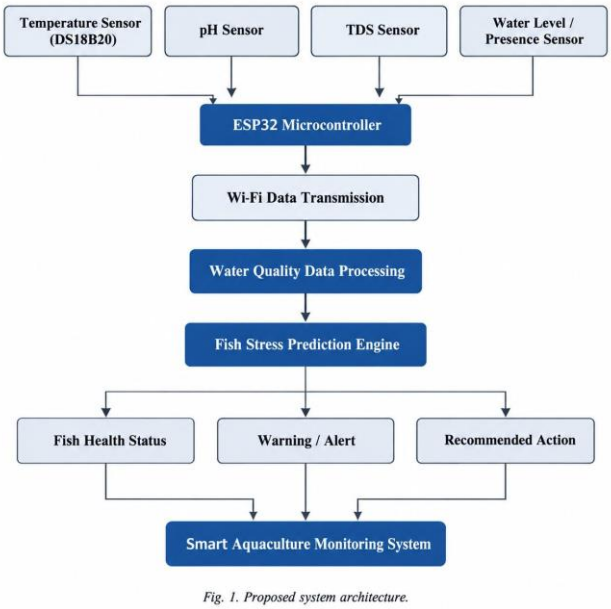


Fig. 1. Proposed system architecture.

Figure 3.1: System Architecture of Smart Aquaculture Assistant

The architecture begins with the sensors. Temperature is measured using a DS18B20 sensor. The water sensor provides a raw wetness/presence signal. The TDS module provides an analog voltage. The pH interface provides an analog signal. These values are read by the ESP32 and printed using fixed serial labels.The Python application opens COM5 at 115200 baud, receives the serial messages and updates the Streamlit dashboard. The dashboard then applies the configured status rules and calculates an overall health status.

3.4 Overall Workflow

Step	Process	Output
1	Sense water conditions	Raw sensor readings
2	Read sensors using ESP32	Temperature, water raw value, TDS voltage, pH
3	Transmit through USB serial	Formatted text messages
4	Parse messages in Python	Numerical values
5	Apply sensor status rules	Normal / At Risk / High Risk or water-level status
6	Calculate overall score	Stable / At Risk / High Risk
7	Display on dashboard	Live monitoring interface
8	Store/train historical ML separately	Random Forest model

Table 3.1: Overall Workflow

3.5 System Modules

Module	Function
Sensor Acquisition	Collects temperature, pH, TDS voltage and water-sensor readings.
ESP32 Controller	Reads sensors and prints standardized serial messages.
Serial Communication	Transfers readings from ESP32 to computer at 115200 baud.
Data Parser	Converts serial strings into numerical values.
Status Engine	Checks sensor readings against prototype ranges.
Health Decision	Calculates the overall Stable/At Risk/High Risk result.
ML Module	Trains and evaluates the historical Random Forest classifier.
Dashboard	Displays live measurements and status information.

Table 3.2: Major System Modules

3.6 Data Flow

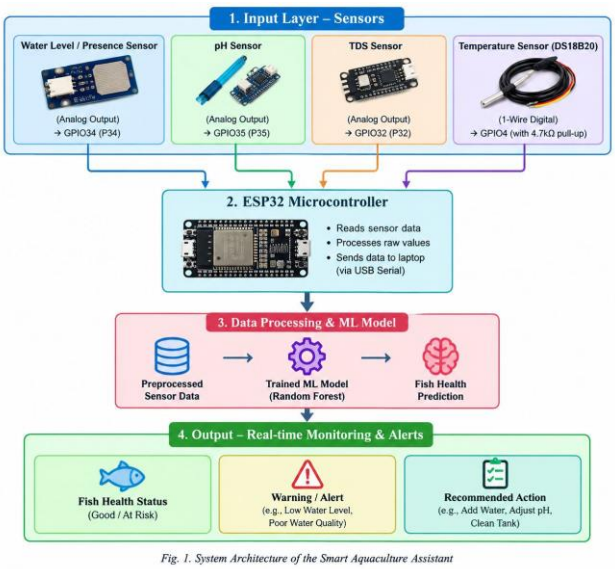


Fig. 1. System Architecture of the Smart Aquaculture Assistant

Figure 3.2: Overall Data Flow

The live data flow is unidirectional during the current prototype: sensor → ESP32 → USB serial → Python parser → status engine → Streamlit dashboard. The historical ML flow is separate: dataset → preprocessing → train/test split → Random Forest → evaluation → saved model.

3.7 Hardware Requirements

Component	Specification / Purpose
ESP32 Dev Module	Main microcontroller for sensor acquisition and serial communication.
DS18B20	Digital temperature sensor; data connected to GPIO4 with pull-up resistor.
pH Sensor Interface	Analog pH signal connected to GPIO35; requires calibration.
TDS Meter V1.0	Analog TDS-related module; voltage read through GPIO34.
HW-038 Water Sensor	Water presence/wetness sensor; signal connected to GPIO32.
Breadboard and Jumper Wires	Prototype wiring and connections.
USB Cable	Power, programming and serial communication.

Table 3.3: Hardware Requirements

3.8 Software Requirements

Software	Purpose
Arduino IDE 2.3.10	ESP32 programming and serial testing.
Python	Application and ML development.
Jupyter Notebook	Dataset analysis and Random Forest training.
Streamlit	Live dashboard.
pandas / NumPy	Data handling and numerical processing.
scikit-learn	Random Forest model training and evaluation.
joblib	Saving the trained model.
pyserial	Reading ESP32 serial data.
Git / GitHub	Version control and source-code submission.

Table 3.4: Software Requirements

3.10 Detailed System Workflow

3.10.1 End-to-End Data Flow

The working sequence begins when the ESP32 is powered and initializes the connected sensors. The DS18B20 is requested for a temperature reading. The analog water sensor, TDS interface and pH interface are then sampled. The ESP32 prints the readings using fixed labels. The Streamlit application opens the same serial port and reads the lines. Python extracts the numeric values and stores the latest readings for display.

The dashboard then evaluates each parameter. Temperature is compared with the prototype temperature ranges. pH is compared with the selected safe range. TDS voltage is treated as a prototype indicator rather than a calibrated ppm measurement. The water sensor value is interpreted using a threshold that must be calibrated for the actual physical setup. Individual statuses are combined to generate Stable, At Risk or High Risk.

3.10.2 Processing Sequence

Sensor initialization → sensor acquisition → serial formatting → serial transmission → Python parsing → dashboard display → parameter assessment → overall status → operator action. This sequence makes the project easy to demonstrate because every stage can be observed separately.

3.10.3 Failure Handling

If the serial port is unavailable, the dashboard reports that the ESP32 is not connected. If the serial monitor is already using COM5, Streamlit cannot open the port. If a sensor returns an invalid value, the reading should be treated cautiously rather than converted into a false precise measurement.

Hardware Integration Overview

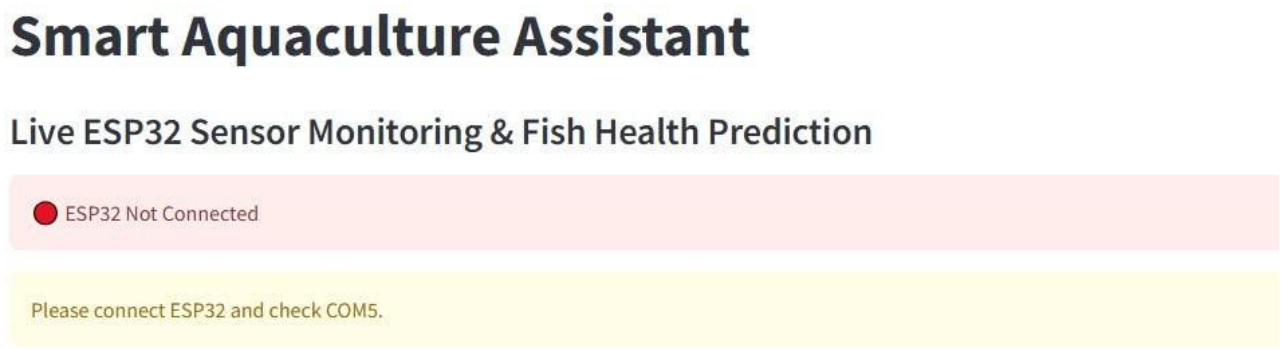


Figure A.1: ESP32 and sensor hardware used for the prototype.

The hardware layer is the physical input stage of the project. The ESP32 acts as the controller and serial communication device. The DS18B20 measures temperature digitally. The HW-038 provides a water-presence or wetness indication. The TDS interface produces an analog voltage that is read by the ESP32. The pH interface is connected to an analog input and requires calibration before reliable field interpretation. The photograph should be replaced with the clearest final project hardware photograph if a newer one is available.

3.11 Sensor Wiring and Pin Mapping

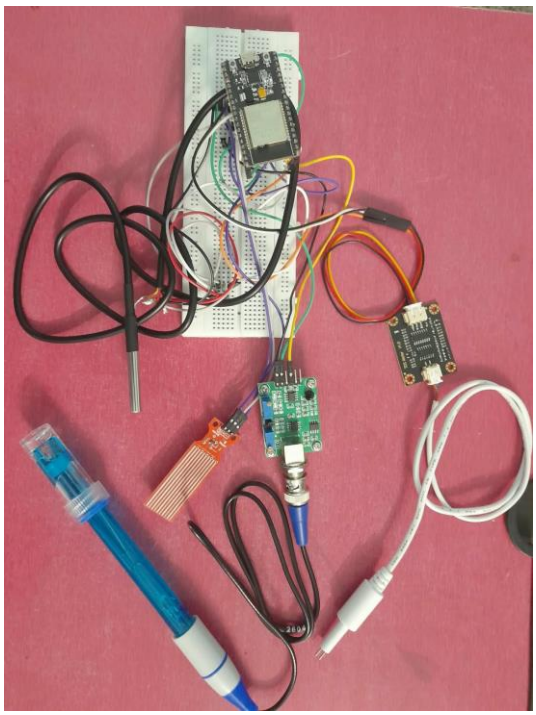


Figure 3.11: Prototype wiring and sensor connections.

The current pin allocation is DS18B20 on GPIO4, HW-038 water sensor on GPIO32, TDS analog output on GPIO34 and pH analog output on GPIO35. The DS18B20 requires a pull-up resistor between the data line and 3.3 V. Analog sensors must share a suitable ground reference with the ESP32. During final hardware assembly, wires should be labelled so that the same pin mapping is used in the Arduino program, report and dashboard.

3.12 Serial Data Acquisition

3.12.1 Serial Protocol

The ESP32 prints a small human-readable protocol at 115200 baud. The important labels are Temperature:, Water Sensor:, TDS Voltage: and pH:. The Streamlit parser searches for these exact prefixes. Keeping the labels unchanged is therefore part of the software interface contract between the Arduino program and Python application.

3.12.2 Example Serial Output

A typical demonstration sequence is: Temperature: 28.25 C; Water Sensor: 2251; TDS Voltage: 0.84 V; pH: 7.01. The values shown here are examples from the prototype testing process and should not be treated as universal calibrated values.

3.12.3 COM Port Operation

The computer must identify the ESP32 on the correct COM port. In the prototype, COM5 was used. Arduino Serial Monitor must be closed before Streamlit opens the port. When the port is occupied, Windows may report that access is denied. The correct recovery is to close the application holding the port, reconnect the ESP32 if necessary, and restart the dashboard.

3.12.4 Advantages of a Fixed Format

A fixed text format is simple for a student prototype because it is easy to inspect manually in Serial Monitor and easy to parse in Python. A later version can replace the text protocol with JSON, timestamped packets or a wireless MQTT/HTTP interface.

3.9 Summary

This chapter described the proposed architecture, workflow, modules, data flow, hardware and software. The design keeps the live sensor prototype and historical machine-learning model clearly separated because their input features are different.

CHAPTER 4
IMPLEMENTATION AND METHODOLOGY

4.1 Dataset Preparation

The project uses the Mendeley aquaculture dataset titled “IoT Monitoring Dataset of Water Quality and Tilapia Health in Aquaculture Ponds in Montería, Colombia”. The project copy is stored as aquaculture-dataset.csv.xlsx. The dataset contains 4,383 observations and 28 columns covering time information, fish information, water-quality parameters, intervention indicators and Health Status.

Item	Value
File	aquaculture-dataset.csv.xlsx
Rows	4,383
Columns	28
Target	Health Status
Stable records	3,678
At Risk records	705
Main ML inputs	Temperature, Dissolved Oxygen, pH, Turbidity

Table 4.1: Dataset Information

4.2 Data Preprocessing

The required columns are selected from the Excel dataset. The model input matrix X contains Temperature (°C), Dissolved Oxygen (mg/L), pH and Turbidity (NTU). The target vector y contains Health Status. The dataset is divided into training and testing subsets using train_test_split with a test size of 20 percent and random_state 42.The preprocessing stage is intentionally simple because the selected Random Forest classifier can operate directly on numerical features. The target labels are retained as Stable and At Risk. Missing or invalid values should be checked before training in a production implementation.

4.3 Random Forest Model

Random Forest is an ensemble classification algorithm that constructs multiple decision trees and combines their predictions. Each tree is trained from a randomized sample of the training data and a randomized subset of features. Combining many trees generally produces a more stable classifier than relying on one decision tree.In this project, 100 trees are used with random_state 42. The model learns relationships between the four selected water-quality features and the historical Health Status label.

4.4 Model Training and Evaluation

The trained model is evaluated using accuracy_score on the test set. The project obtained approximately 99.89% accuracy on the selected test split. This result should be interpreted carefully because the dataset contains time-series-like observations and repeated environmental patterns. A

future study should use time-based validation or a pond-level split to estimate generalization more realistically.

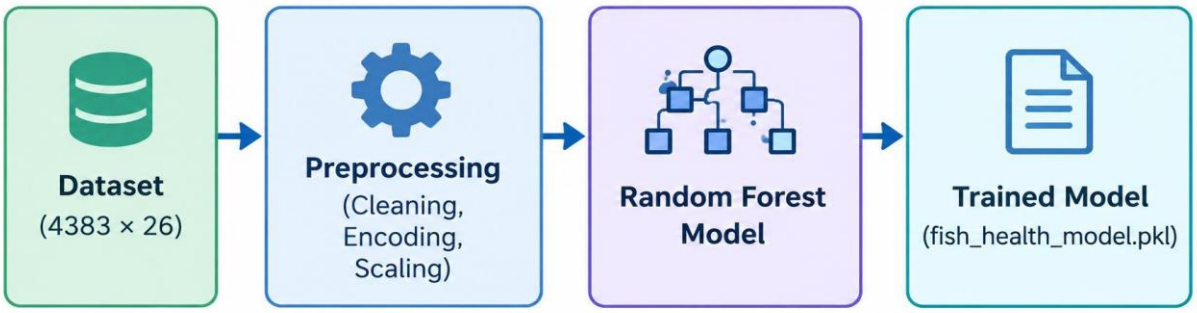


Figure 4.1: Dataset-to-Model Development Workflow

4.5 ESP32 Sensor Implementation

The ESP32 program initializes the DS18B20 sensor library and the analog pins used by the water sensor, TDS module and pH interface. Every two seconds it prints the four measurements using fixed labels. The Streamlit application depends on these labels, so the labels must not be changed without also changing the parser.

Sensor	ESP32 Pin	Output
DS18B20 Temperature	GPIO4	Temperature in °C
HW-038 Water Sensor	GPIO32	Raw analog value
TDS Meter V1.0	GPIO34	TDS-related voltage
pH Interface	GPIO35	Calculated pH estimate

Table 4.2: Live Sensor Parameters



Figure 4.2: ESP32 Connection Diagram

4.6 Streamlit Dashboard

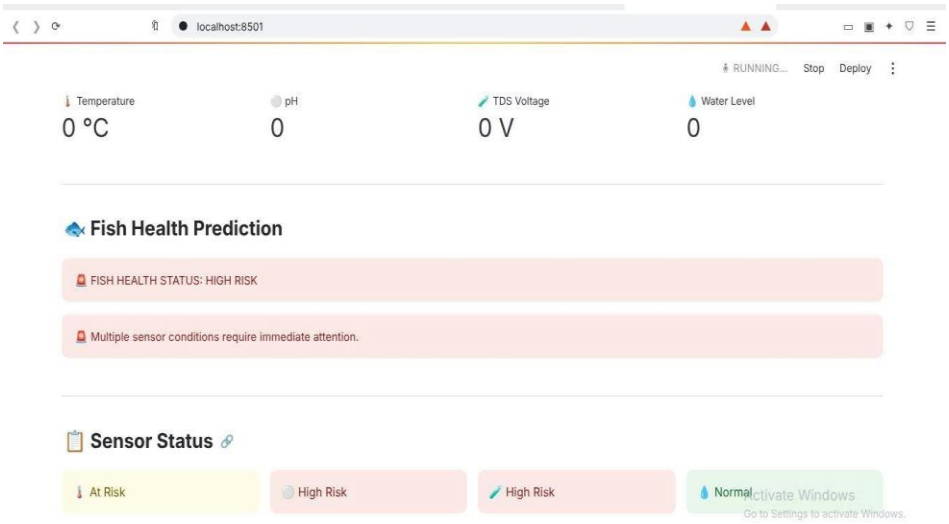


Figure 4.3: Streamlit Dashboard Architecture / Interface

The Streamlit application opens COM5 at 115200 baud. It creates four live metric areas for temperature, pH, TDS voltage and water level. It also creates four sensor-status areas and one overall fish-health prediction area. The application continuously reads serial lines and updates the interface.

4.7 Rule-Based Live Health Logic

The current live prototype uses a transparent rule-based decision layer. Temperature is treated as Normal from 25–30 °C, At Risk from 23–25 °C or 30–32 °C, and High Risk outside these ranges. pH is treated as Normal from 6.5–8.5, At Risk from 6.0–6.5 or 8.5–9.0, and High Risk outside these ranges.For TDS, the current prototype uses voltage ranges rather than calibrated ppm values. A voltage from 0.30–1.50 V is treated as Normal, 0.20–0.30 V or 1.50–1.80 V as At Risk, and values outside those ranges as High Risk. The water sensor uses a prototype threshold of 1000 raw units for Normal, 500–999 for Low, and below 500 for Very Low.The overall score gives one point for each Normal sensor condition. A score of 4 produces STABLE, a score of 2 or 3 produces AT RISK, and a score of 0 or 1 produces HIGH RISK.

4.8 Calibration and Limitations

The pH calculation used in the prototype is based on a simple calibration relationship and should not be treated as a laboratory-grade pH measurement. The TDS module currently reports voltage rather than a validated ppm value. The HW-038 sensor is a water-presence/wetness sensor rather than a calibrated water-depth sensor. These limitations are explicitly documented so that the prototype is not presented as a production aquaculture monitoring instrument.

4.9 GitHub and Reproducibility

The intended GitHub repository structure separates the application, ESP32 code, ML notebook, dataset, saved model, images and requirements. This organization makes it easier for a reviewer to inspect the project and reproduce the software setup.

Folder / File	Purpose
README.md	Project overview, setup and usage.
app.py	Streamlit live dashboard.
requirements.txt	Python dependencies.
ESP32/smart_aquaculture_esp32.ino	ESP32 sensor code.
ML/fish_health_model.ipynb	Dataset analysis and model training.
Dataset/aquaculture-dataset.csv.xlsx	Historical aquaculture dataset.
Model/fish_health_model.pkl	Saved Random Forest model.
Images/	Architecture and dashboard images.

Table 4.3: Project Repository Structure

4.11 Dataset and Data Dictionary

4.11.1 Dataset Overview

The historical dataset used for the machine-learning experiment contains 4,383 rows and 28 columns. Important variables include Datetime, Temperature (°C), Dissolved Oxygen (mg/L), pH, Turbidity (NTU), intervention indicators, derived risk indicators and Health Status. The target labels observed in the dataset are Stable and At Risk.

4.11.2 Main Model Fields

Temperature (°C) represents water temperature. Dissolved Oxygen (mg/L) represents oxygen availability. pH represents acidity or alkalinity. Turbidity (NTU) represents water clarity. Health Status is the classification target. These four physical variables were selected for the Random Forest experiment.

4.11.3 Live-versus-Historical Mapping

The current live prototype measures temperature, water presence, TDS voltage and pH. Therefore the live hardware does not provide the same four variables used by the trained Random Forest model. The report must keep this distinction explicit to avoid claiming that the old model directly predicts live data without the required feature conversion or retraining.

4.11.4 Data Quality Considerations

Before using a dataset for machine learning, missing values, duplicate records, incorrect units, outliers and inconsistent labels should be checked. The prototype project uses the supplied dataset as the experimental basis and records the model result obtained from its train-test split.

4.12 Machine Learning Implementation

4.12.1 Model Preparation

The machine-learning workflow begins by loading the Excel dataset using pandas. The four selected columns are separated into the feature matrix X and Health Status is assigned to y. The dataset is divided into training and testing subsets using train_test_split with test_size=0.2 and random_state=42.

4.12.2 Random Forest Classifier

A RandomForestClassifier with 100 estimators and random_state=42 is trained on the training subset. Random Forest is suitable for a student prototype because it can model nonlinear relationships and can handle multiple numeric features without requiring a complex mathematical formulation.

4.12.3 Prediction and Evaluation

The trained model predicts the test samples and accuracy_score compares predicted and actual labels. The project recorded approximately 99.89% accuracy on the selected test split. This result is reported as an experimental dataset result, not as certified real-world farm accuracy.

4.12.4 Model Saving

The trained estimator is stored as fish_health_model.pkl using joblib.dump. The saved file can then be loaded by another Python program without retraining the model. A future version should store the training feature names and preprocessing steps alongside the model to reduce the chance of feature mismatch.

4.13 Model Evaluation and Interpretation

4.13.1 Meaning of Accuracy

Accuracy is the proportion of correctly classified samples in the test set. Although the project obtained 99.89%, a high accuracy score alone does not prove that a model is suitable for all ponds or seasons. Class balance, leakage, data collection conditions and the similarity between training and real-world data must also be considered.

4.13.2 Confusion-Matrix Perspective

A confusion matrix can show how many Stable samples were correctly classified and how many At Risk samples were missed or incorrectly flagged. For aquaculture monitoring, false negatives can be important because a missed risk condition may delay corrective action. Therefore future evaluation should include precision, recall, F1-score and class-specific analysis.

4.13.3 Practical Interpretation

The model result demonstrates the feasibility of applying supervised machine learning to the historical dataset. It does not remove the need for sensor calibration, domain knowledge or human

supervision. The dashboard should therefore be treated as a decision-support tool that provides an early indication rather than an autonomous veterinary or farm-management authority.

4.13.4 Reproducibility

The notebook, dataset, saved model and requirements file should be stored in the GitHub project structure so that the experiment can be repeated. The same random_state and feature selection should be retained when comparing future versions.

4.14 Streamlit Dashboard Design

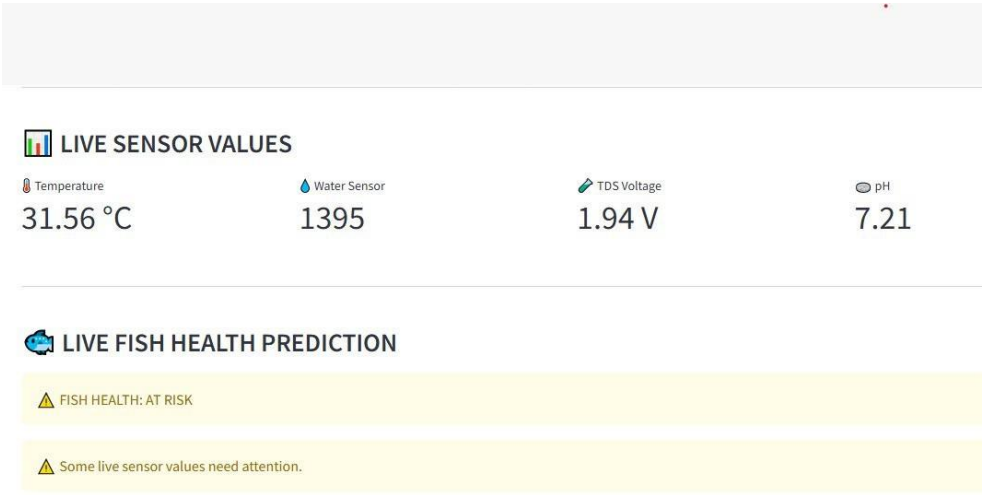


Figure 4.14: Streamlit live monitoring dashboard.

The Streamlit interface is the user-facing software layer. It presents the latest sensor values, connection status and overall condition. A simple dashboard is useful during project demonstration because the evaluator can observe the ESP32 output and the corresponding Python interpretation. The final screenshot should show the actual working dashboard with the project title and all available live parameters.

4.15 ESP32 Connection and Pin Details

B.1 Purpose of the Connection Diagram

The ESP32 is the central controller of the live Smart Aquaculture Assistant prototype. The connection diagram should be placed in Chapter 4 as Figure 4.2 because it helps the evaluator understand the physical relationship between the sensors and the controller before reading the software implementation. The appendix provides the detailed written mapping so that the diagram can be checked against the actual hardware.

B.2 Final Pin Mapping

The DS18B20 temperature sensor uses GPIO4 for its data line. Its red wire is connected to 3.3 V and black wire to GND. A 4.7 kΩ pull-up resistor is used between the data line and 3.3 V. The HW-038 water-presence sensor uses GPIO32 as its signal input. The TDS interface output is connected to GPIO34, while the pH interface output is connected to GPIO35.

Table B.1: ESP32 Pin Mapping

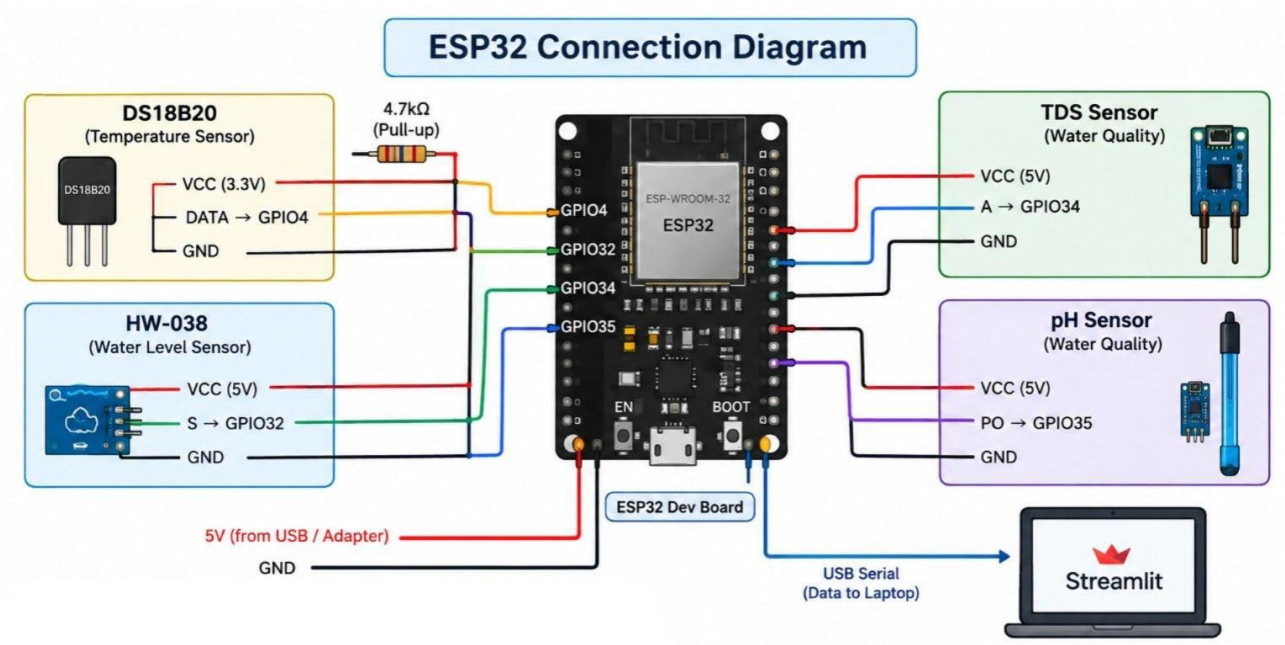
DS18B20 | DATA → GPIO4 | Temperature measurement | 4.7 kΩ DATA-to-3.3 V pull-up

HW-038 | S → GPIO32 | Water presence / wetness indication | Common GND required

TDS Meter V1.0 | A → GPIO34 | Analog TDS voltage | Prototype voltage indication

pH Interface | PO → GPIO35 | Analog pH signal | Requires calibration

B.3 Image to Add



B.4 Wiring Verification

Before powering the system, each sensor connection should be compared with the diagram and the Arduino source code. The sensor ground lines should share a suitable common reference with the ESP32. The final photograph should show the physical wiring clearly enough for an evaluator to compare it with Figure 4.2.

4.16 Calibration, Limitations and Validity

4.16.1 Sensor Calibration

Sensor calibration is essential for field deployment. A pH sensor normally requires buffer-solution calibration and stable electrical connections. A TDS sensor requires a known calibration solution if the final system is expected to display ppm or conductivity-related values. A wetness sensor needs a physical threshold determined by the tank or container geometry.

4.16.2 Current Prototype Limitations

The present prototype does not contain a calibrated dissolved-oxygen sensor and does not measure turbidity live. The HW-038 is used as a water-presence indicator rather than a calibrated water-level percentage sensor. TDS is currently displayed as voltage, not as a validated ppm value. The pH conversion is a prototype formula and should not be interpreted as laboratory-grade measurement.

4.16.3 Model Limitation

The Random Forest model was trained with Temperature, Dissolved Oxygen, pH and Turbidity. Because the live prototype has a different sensor set, the model should not be directly connected to the current live inputs without retraining or an appropriate feature-engineering strategy. This limitation is documented intentionally for technical correctness.

4.16.4 Future Validation

Future work should collect synchronized live measurements, calibrate all sensors, label fish-health conditions with expert or farm observations, retrain the model using the live feature set, and validate performance on an independent pond or time period.

4.10 Summary

This chapter described dataset preparation, model training, sensor implementation, dashboard development, rule-based health assessment, calibration limitations and repository organization.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Dataset Results

Health Status	Count	Percentage
Stable	3,419	78%
At Risk	438	10%
High Risk	526	12%
Total	4,383	100%

Table 5.1: Health Status Distribution

The dataset is dominated by Stable observations. This class imbalance is important when interpreting accuracy because a classifier that strongly favours the majority class can still obtain a high accuracy. Therefore, future work should include precision, recall, F1-score and confusion matrix analysis, along with time-aware validation.

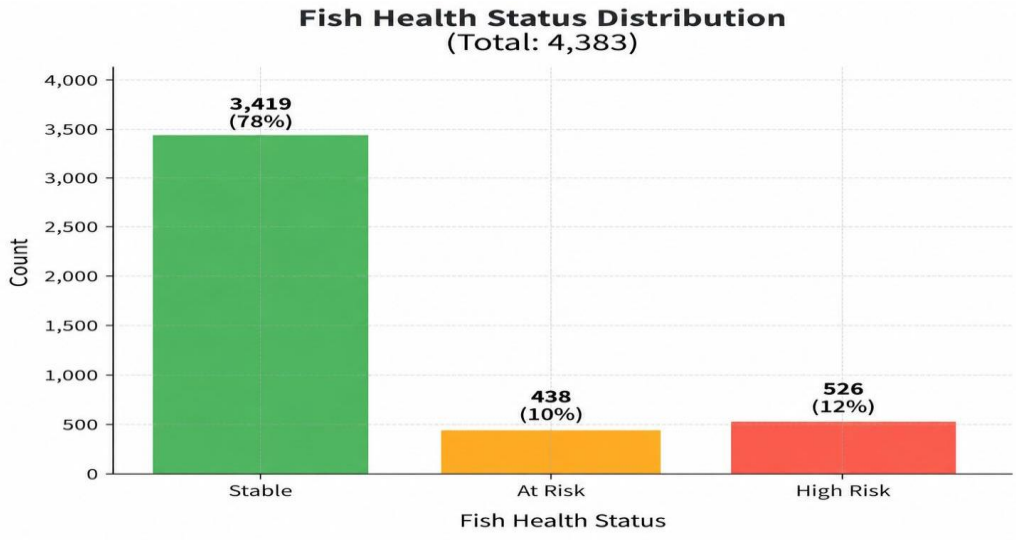


Figure 5.1: Health Status Distribution in Dataset

5.2 Model Results

Metric	Project Result
Algorithm	Random Forest Classifier
Number of trees	100
Train/Test split	80% / 20%
Random state	42
Features	Temperature, DO, pH, Turbidity
Test accuracy	99.89%

Table 5.2: Random Forest Model Evaluation

The model achieved approximately 99.89% accuracy on the project test split. The model also correctly predicted the sample input Temperature 27.5 °C, Dissolved Oxygen 6.3 mg/L, pH 8.0 and Turbidity 3.3 NTU as Stable. The high accuracy is encouraging for the academic prototype, but it should not be interpreted as field-level accuracy without independent validation.

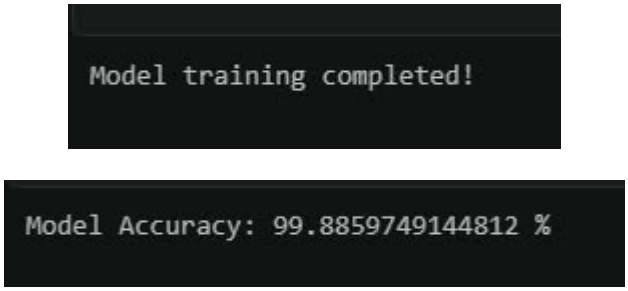


Figure 5.2: Random Forest Model Evaluation

5.3 Sensor Results

Test	Observed Example	Interpretation
Temperature	28.25 °C	Within prototype normal range
TDS voltage	About 0.79–0.84 V in tap water	Prototype voltage reading; ppm not calibrated
Water sensor	Raw values varied with wet/dry condition	Useful for presence/level indication
pH	Previously around 7.0 during testing	Requires proper calibration for reliable field use

Table 5.3: Sample Sensor Test Results

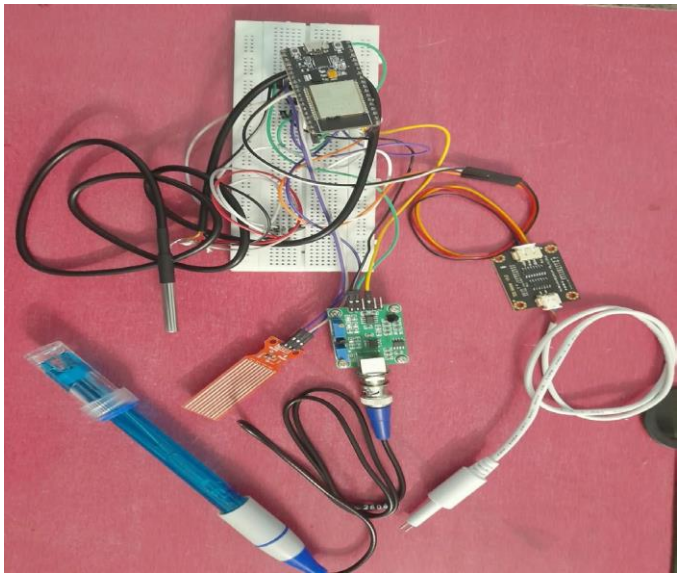


Figure 5.3: ESP32 Hardware Prototype

The hardware prototype successfully demonstrated sensor acquisition and serial output. Sensor values should be recorded with date, water type and calibration condition during future testing so that meaningful calibration curves can be created.

5.4 Dashboard Results

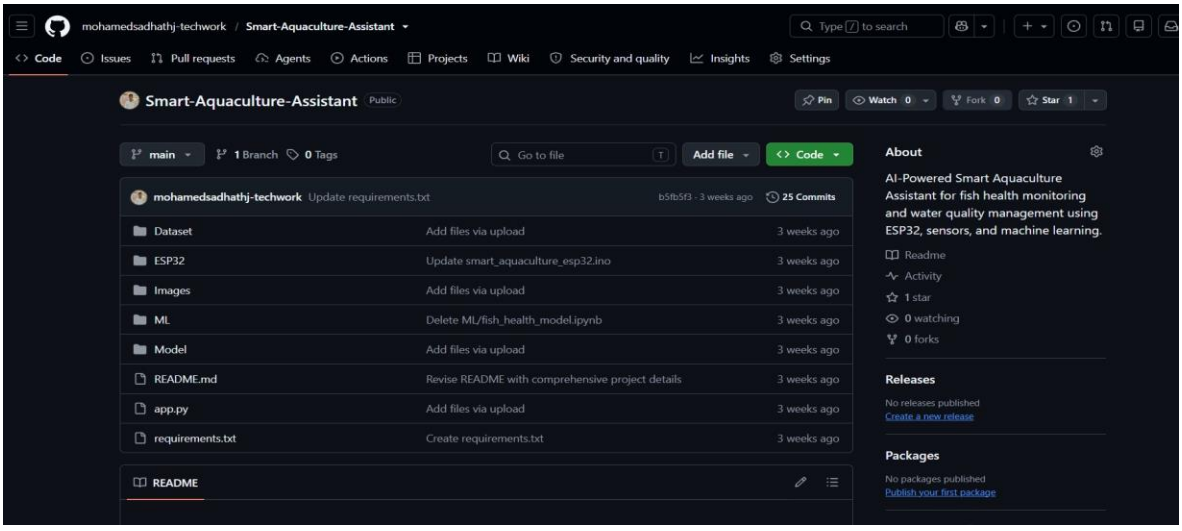


Figure 5.4: GitHub Repository Structure

The dashboard is designed to show four live measurements, sensor-level status and an overall fish-health status. A normal combination of all four conditions results in STABLE. When several conditions move outside the normal range, the dashboard changes the overall status to AT RISK or HIGH RISK.

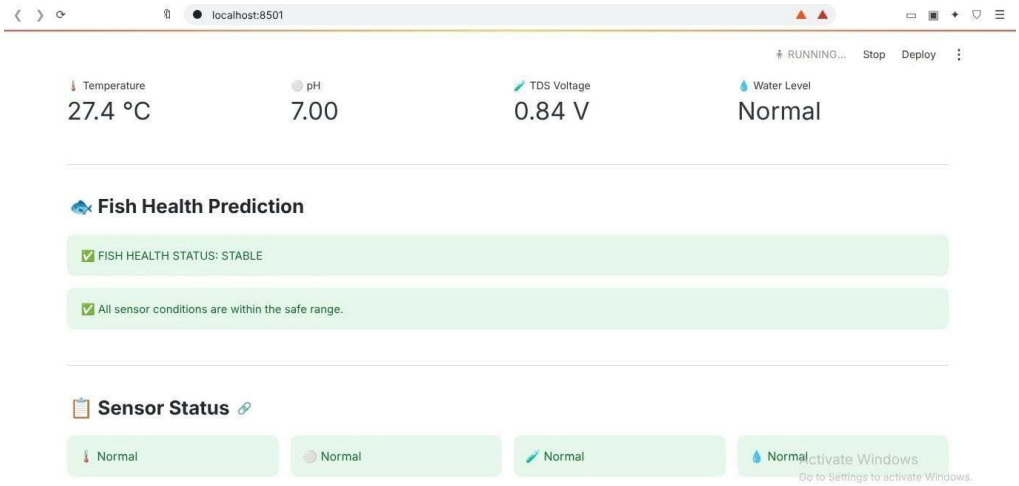


Figure 5.5: Final Streamlit Dashboard Output

5.5 Functional Testing

Test Case	Expected Result	Status
ESP32 connection	COM5 opens at 115200 baud	Passed during testing
Temperature reading	Temperature value appears	Passed
Water sensor reading	Raw water value appears	Passed
TDS reading	Voltage value appears	Passed
pH reading	pH value appears when valid signal is available	Prototype tested
Normal conditions	Overall status becomes Stable	Implemented
Mixed conditions	Overall status becomes At Risk	Implemented
Multiple abnormal conditions	Overall status becomes High Risk	Implemented
Dashboard parsing	Serial labels converted to values	Implemented
GitHub organization	Files separated into folders	Implemented

Table 5.4: Functional Testing Summary

5.6 Discussion

The project demonstrates that a low-cost embedded controller can be used to collect multiple water-related measurements and present them in a simple dashboard. The most important achievement is the integration of hardware, software and machine learning into one educational prototype.The ML result demonstrates the feasibility of classifying historical fish-health observations using Random Forest. However, the live dashboard currently uses different inputs. This separation is technically important: the old model expects Dissolved Oxygen and Turbidity, while the live

prototype reads TDS and a water sensor. A future retraining stage should create a model whose feature set exactly matches the calibrated live sensors.

5.7 Comparison with Existing Approaches

Feature	Manual Monitoring	Typical IoT Monitoring	Proposed Prototype
Continuous sensing	No	Yes	Yes
Live dashboard	No	Usually	Yes
Health status	Manual interpretation	Sometimes	Yes
Historical ML	No	Optional	Yes
Low-cost educational prototype	Yes	Varies	Yes
Field validation	Depends	Depends	Not yet completed

Table 5.5: Comparison with Existing Approaches

5.9 Software Test Cases

Test ID	Input / Condition	Expected Result	Status
TC01	ESP32 connected, valid temperature	Temperature value appears	Pass
TC02	Water sensor changes state	Raw value changes	Pass / Calibrate
TC03	TDS connected	Voltage value displayed	Pass
TC04	pH signal connected	Raw/conversion displayed	Needs calibration
TC05	Serial Monitor open	Streamlit cannot access COM5	Expected behavior
TC06	All sensors active	Dashboard shows all fields	Pass when parser matches labels
TC07	Normal parameter values	Overall status can become Stable	Pass
TC08	Multiple abnormal values	Overall status becomes High Risk	Pass
TC09	Missing serial data	Connection warning displayed	Required

The test cases are designed around the actual prototype failure modes observed during development. In particular, COM-port occupation, pH electrical behavior and sensor calibration are treated as engineering issues rather than hidden from the evaluator.

5.10 Results and Sample Outputs

5.10.1 Sensor Demonstration Results

During prototype testing, the DS18B20 produced realistic temperature values such as 28.25 °C. The TDS interface produced an analog voltage around 0.84 V in one tap-water test. The water sensor produced raw analog values that changed with its physical condition. The pH interface produced inconsistent values during some tests, including a raw reading of 4095 when the signal behaved as a full-scale input. This confirms that calibration and electrical verification are necessary before claiming accurate pH measurement.

5.10.2 Dashboard Interpretation

The dashboard can display each parameter status independently. For example, a temperature slightly above the normal range can be labelled At Risk, while a strongly abnormal pH can be labelled High Risk. The overall result combines these individual conditions. The final demonstration should show at least one normal/stable case and one risk case so that the evaluator can see that the logic changes with input.

5.10.3 Evidence to Include

The strongest result section should contain the final hardware photograph, Serial Monitor screenshot, Jupyter Notebook accuracy output, model prediction output, Streamlit dashboard screenshot, GitHub repository screenshot and system architecture diagram. These images provide direct evidence that the report describes an implemented prototype rather than only a proposed design.

5.11 Sensor Testing and Calibration Record

C.1 Testing Method

The sensors were tested individually before being used together. The ESP32 Serial Monitor was used to observe raw and converted values. After individual testing, the sensors were integrated into the combined program so that the Streamlit application could read the fixed serial labels.

C.2 Temperature Sensor Test

The DS18B20 produced realistic temperature readings during prototype testing, including a value around 28.25 °C. This indicates that the digital sensor and GPIO4 connection were functioning during the test.

C.3 TDS Sensor Test

The TDS interface produced an analog voltage around 0.84 V during one tap-water test. The current prototype intentionally reports voltage rather than claiming a calibrated TDS value in ppm.

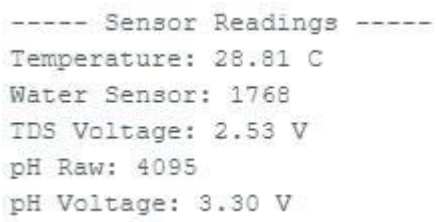
C.4 Water Sensor Test

The HW-038 sensor produced changing raw analog readings depending on its physical condition. Because it is not a calibrated depth sensor, the current application treats its value as a prototype water-presence/level indicator using a threshold.

C.5 pH Sensor Test

The pH interface produced useful readings in some tests but also showed full-scale raw values such as 4095 during an unstable or disconnected condition. This demonstrates why pH calibration, stable power, correct grounding and proper probe placement are important.

C.6 Image to Add



C.7 Calibration Recommendation

For future deployment, pH should be calibrated with standard buffer solutions, TDS should be calibrated using an appropriate reference solution, and the water sensor threshold should be determined experimentally. Sensor values should be logged with timestamps so that unstable readings can be identified.

5.14 Reliability, Safety and Security

5.14.1 Reliability Measures

The ESP32 should be powered from a stable supply and all sensor grounds should be common. Wires should be secured to avoid intermittent readings. Software should handle missing serial data and invalid sensor values without crashing. The dashboard should display a connection warning when no recent data is received.

5.14.2 Data Integrity

The serial labels must remain consistent with the Python parser. A timestamp can be added to each packet in a future version so that stale values can be detected. Logging the raw sensor readings before conversion would also help diagnose calibration problems.

5.14.3 Security Considerations

The current USB serial prototype has a limited attack surface compared with a network-connected system. If Wi-Fi and cloud services are added, authentication, encrypted communication, access control, secret management and safe database configuration will become necessary.

5.14.4 Safe Demonstration Practice

The hardware should be operated with appropriate voltage levels for each sensor. Connections should be checked before powering the ESP32. Water should not be allowed to reach exposed electrical connections. The project is a monitoring prototype and should not be connected directly to pumps, feeders or other high-power equipment without suitable isolation and safety controls.

5.8 Summary

5.15 Integrated Validation Summary

The integrated validation of the Smart Aquaculture Assistant considered the hardware layer, serial communication layer, software dashboard layer and offline machine-learning layer separately. This separation is important because each layer has different inputs, outputs and limitations. The ESP32 layer was checked by observing whether the connected sensors produced readable values through the Arduino Serial Monitor. The software layer was checked by confirming that the Streamlit application could open the configured serial port, identify the expected labels and update the dashboard values. The machine-learning layer was evaluated independently using the historical aquaculture dataset and a fixed train-test split.The hardware tests showed that the DS18B20 temperature sensor was the most stable of the implemented sensors. During testing it produced realistic values such as approximately 28.25 °C, demonstrating that the digital one-wire communication and GPIO4 connection were functioning. The TDS interface also produced an analog voltage value, with approximately 0.84 V observed during one tap-water test. Because the TDS sensor was not calibrated against a reference solution, the prototype reports voltage rather than claiming an exact TDS concentration in parts per million. The HW-038 water sensor produced changing analog values, but it is treated as a water-presence or wetness indicator rather than a calibrated depth sensor.The pH interface required additional attention. Some tests produced useful raw values, while other tests produced a full-scale reading of 4095, corresponding to approximately 3.3 V at the ESP32 analog input. Such a result can occur when the signal is disconnected, saturated or electrically unstable. The report therefore does not present the pH measurement as field-calibrated. This observation demonstrates why sensor calibration and electrical verification must be completed before a low-cost prototype is used for real farm decisions.

Software validation also considered practical failure conditions. The Streamlit application must not share COM5 with the Arduino Serial Monitor at the same time. When the serial port is occupied, the dashboard cannot receive live values and reports a connection problem. After the Serial Monitor is closed, the application can access the port and parse the four expected serial labels: Temperature, Water Sensor, TDS Voltage and pH. The live decision layer then assigns individual conditions and produces the overall Stable, At Risk or High Risk result according to the configured prototype rules.The offline Random Forest experiment provides a separate quantitative result. The historical dataset contains 4,383 observations and 28 columns, and the selected features are Temperature, Dissolved Oxygen, pH and Turbidity. The model achieved approximately 99.89% accuracy on the selected test split. This result supports the feasibility of supervised classification on the historical data, but it does not prove that the same accuracy will be obtained from the current live hardware. The difference between the historical model features and the live sensor features is therefore explicitly treated as a limitation and a future improvement area.

5.16 Practical Interpretation and Engineering Lessons

The practical results of the project show that an IoT monitoring system should be designed as a sequence of dependable layers rather than as a single application. The first layer is physical sensing. Sensors must be connected to suitable voltage levels, share a reliable ground reference and be checked individually before combined operation. The second layer is data acquisition. The ESP32 must read each sensor at a suitable interval and transmit clearly labelled values. The third layer is communication. The computer application must open the correct serial port and interpret the transmitted labels without ambiguity. The fourth layer is visualization and decision support, where the latest values are converted into understandable status information for the user.One important lesson from the project is the value of transparent status rules during prototype development. The current dashboard does not hide the decision process behind an unexplained output. Temperature, pH, TDS voltage and the water-sensor reading are shown separately, and each parameter is assigned a condition. The overall result is then derived from the number of parameters considered normal. Four normal conditions produce Stable, two or three produce At Risk, and zero or one produces High Risk. This approach is simple enough to demonstrate during a project review and makes troubleshooting easier because the user can identify which parameter caused the overall status to change.

The project also demonstrates why data consistency is essential in machine-learning applications. A model can only be used correctly when the live input variables have the same meaning, units and feature order as the variables used during training. The historical Random Forest model was trained using Temperature, Dissolved Oxygen, pH and Turbidity, while the live prototype currently measures Temperature, pH, TDS voltage and a water-sensor signal. Directly passing the live values into the historical model would therefore be technically incorrect. The current design avoids this mismatch and keeps the offline ML experiment separate from the live rule-based monitoring prototype.From a software engineering perspective, reproducibility is another important outcome. The project keeps the ESP32 sketch, Streamlit application, requirements file, dataset, notebook and saved model in a structured project directory. The serial labels used by the ESP32 are documented because a small change in a label can prevent the Python parser from recognizing a value. The same principle applies to the COM port and baud rate. Documenting these dependencies makes the project easier for another student or evaluator to reproduce.Finally, the prototype should be considered an educational and engineering proof of concept rather than a production aquaculture control system. Before deployment in a real pond, the sensors should be calibrated, additional water-quality variables should be measured, the model should be retrained using synchronized live variables and expert-labelled fish-health observations, and the complete system should be evaluated over longer periods. With these improvements, the Smart Aquaculture Assistant can evolve from a demonstration prototype into a more robust decision-support platform for practical aquaculture management.

CHAPTER 6
LIVE SYSTEM DEMONSTRATION

6.1 Overview

The live demonstration consists of three stages: hardware initialization, serial data acquisition and Streamlit visualization. The ESP32 is connected to the computer using USB. Arduino Serial Monitor is closed before the Streamlit application opens COM5, because two applications cannot normally use the same serial port simultaneously.

6.2 ESP32 Live Monitoring

```
----- Sensor Readings -----
Temperature: 28.81 C
Water Sensor: 1768
TDS Voltage: 2.53 V
pH Raw: 4095
pH Voltage: 3.30 V
```

Figure 6.1: Live ESP32 Sensor Output

The ESP32 prints a repeating block of sensor information. The fixed prefixes Temperature:, Water Sensor:, TDS Voltage: and pH: allow the Streamlit parser to identify each value.

6.3 Streamlit Live Dashboard

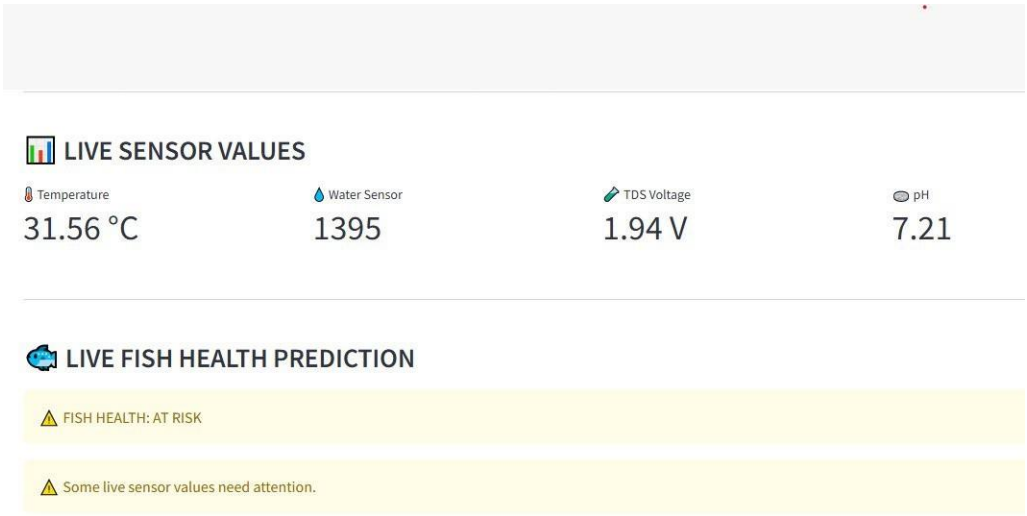


Figure 6.2: Stable Dashboard Output

The dashboard provides a simple user interface with four live metric cards. The sensor status section shows the condition of each measurement. The fish-health section provides the overall decision, helping the user understand the combined state without manually comparing all numerical values.

6.4 Sample Stable Case

A Stable case occurs when all four sensor conditions fall inside the configured normal ranges. For example, a temperature near 28 °C, pH near neutral-to-slightly-alkaline range, TDS voltage inside the prototype normal voltage range and a water-sensor reading above the configured threshold can produce four normal conditions. The score becomes 4 and the dashboard displays STABLE.

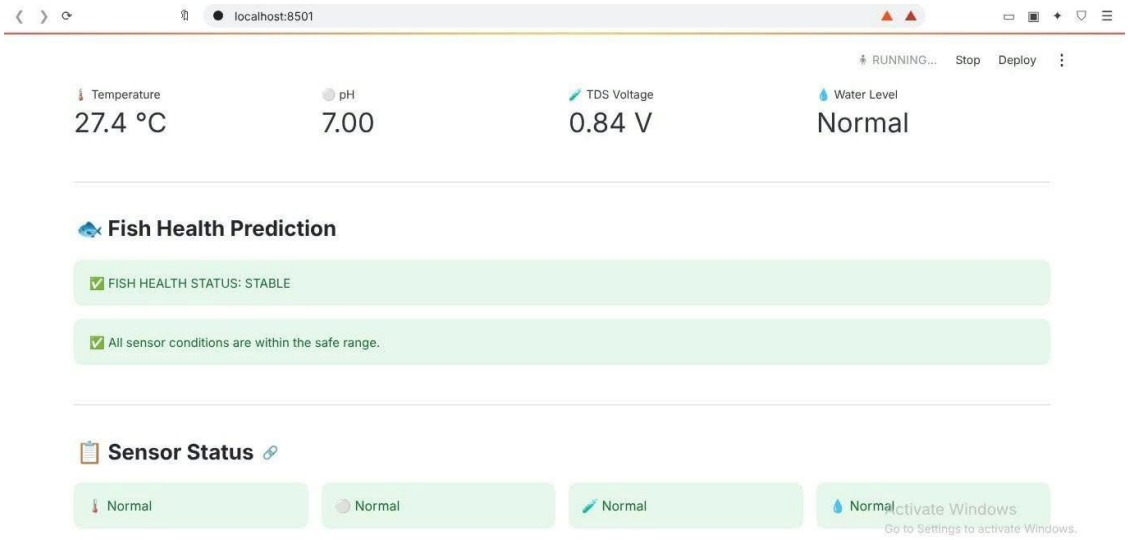


Figure 6.3: Stable Fish Health Dashboard

6.5 Sample At-Risk Case

An At-Risk case occurs when two or three of the four conditions are normal while one or more conditions require attention. The dashboard displays warning indicators for the affected sensors and shows AT RISK as the overall status.

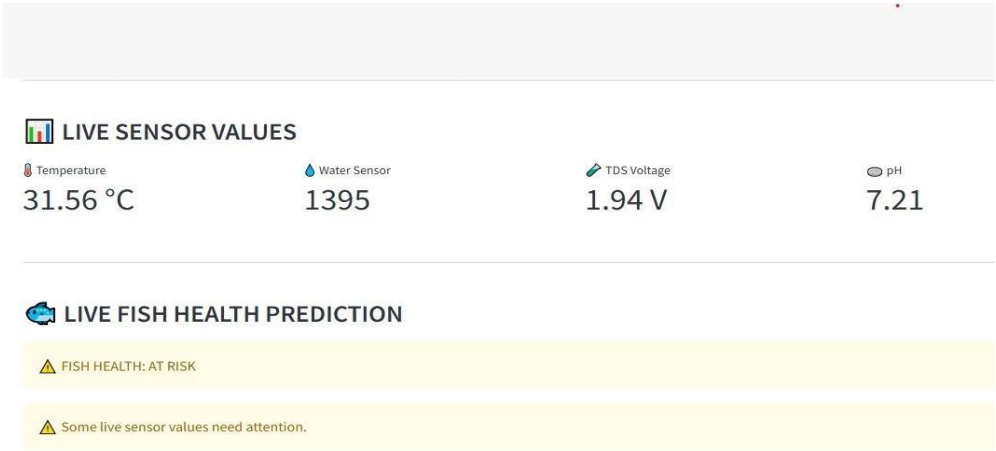


Figure 6.4: At-Risk Dashboard Output

6.6 Sample High-Risk Case

A High-Risk case occurs when zero or one of the four conditions is normal. The dashboard uses the error state to communicate that multiple sensor conditions require immediate attention. The exact thresholds should be replaced with scientifically validated ranges after calibration and consultation with aquaculture experts.

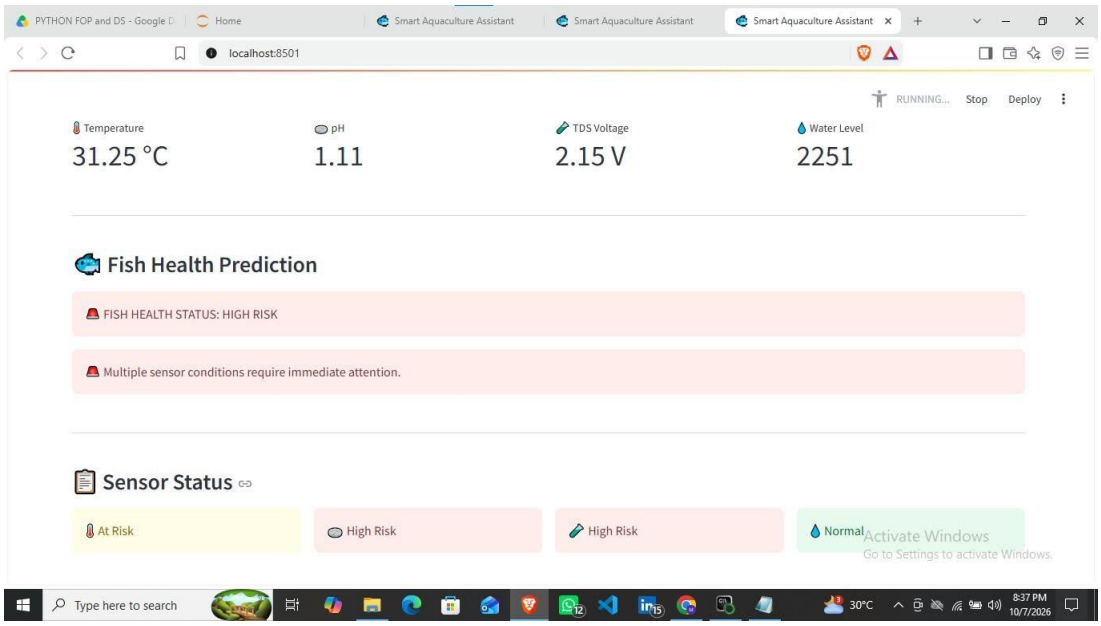


Figure 6.5: High-Risk Dashboard Output

6.8 Deployment and Maintenance Plan

6.8.1 Deployment Procedure

First assemble the sensors according to the documented pin map. Upload the Arduino sketch. Confirm readings in Serial Monitor. Close Serial Monitor. Install Python dependencies from requirements.txt. Place app.py and the required model files in the project directory. Run streamlit run app.py and verify that the dashboard receives the serial values.

6.8.2 Maintenance Schedule

Before each demonstration, inspect wiring and confirm the COM port. Periodically inspect the pH and TDS probes, clean them according to their manufacturer guidance, and recalibrate when necessary. Back up the dataset, notebook, model and source code before making major changes.

6.8.3 Software Maintenance

When changing the Arduino serial labels, update the Streamlit parser at the same time. When changing the machine-learning features, retrain and save a new model and record the feature list in the README. Keep versioned copies of working code so that a failed experiment can be reverted.

6.8.4 Expansion Path

A later version can add Wi-Fi, a database, cloud storage, notification alerts, calibrated DO and turbidity sensors, automatic pump control with electrical isolation, time-series prediction and a retrained model that matches the live sensor feature set.

6.9 User Operation Manual

6.9.1 Starting the System

Connect the ESP32 and sensors. Open Arduino IDE and select the ESP32 board and the correct COM port. Upload the final sketch. Open Serial Monitor at 115200 baud and verify that all four sensor labels appear. Close Serial Monitor before opening Streamlit.

6.9.2 Starting the Dashboard

Open Command Prompt or VS Code in the project directory. Install the packages listed in requirements.txt if they are not already installed. Run streamlit run app.py. The browser should open the Smart Aquaculture Assistant dashboard. The connection indicator should show that live data is being received.

6.9.3 Reading the Dashboard

Observe temperature, water-sensor value, TDS voltage and pH. Each value should be interpreted together with its status. The overall status is intended to provide a quick summary. A Stable result indicates that the prototype rules currently consider the measured conditions normal. At Risk indicates that one or more conditions need attention. High Risk indicates that the combined score is strongly abnormal.

6.9.4 Shutdown

Stop the Streamlit application after demonstration, disconnect power safely, and store the sensors dry and protected as appropriate. Save any screenshots or logs required for the project evidence.

6.10 Demonstration Guide

F.1 Starting the Hardware

Connect the ESP32 to the computer and confirm the correct board and COM port. Upload the combined sensor program. Open Serial Monitor at 115200 baud and confirm the four expected labels: Temperature:, Water Sensor:, TDS Voltage: and pH:.

F.2 Starting the Dashboard

Close Arduino Serial Monitor before starting Streamlit because both applications cannot normally use COM5 at the same time. Open Command Prompt or VS Code in the project directory and run streamlit run app.py.

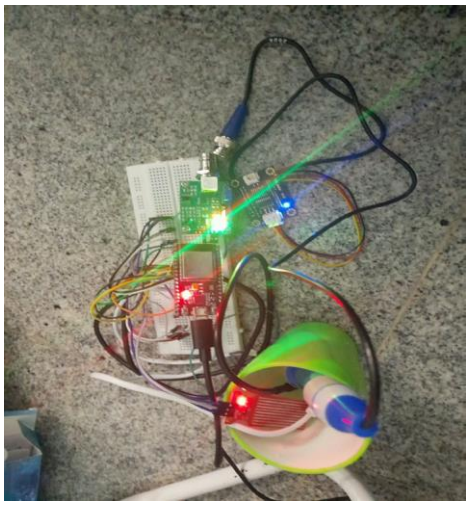
F.3 Demonstration Sequence

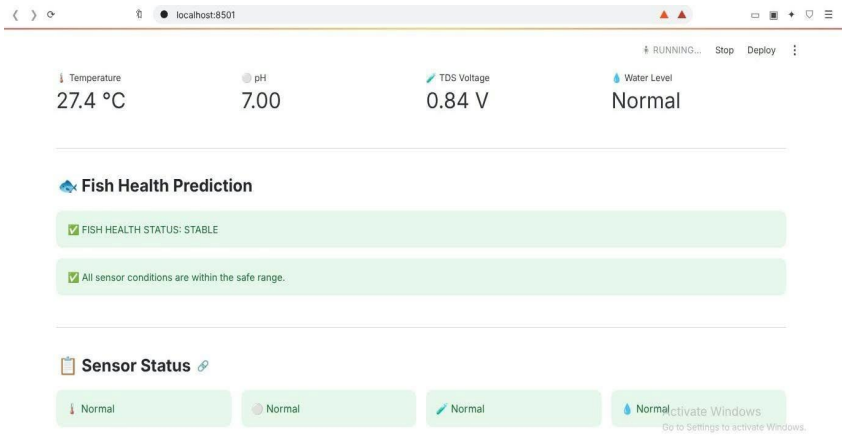
First show the ESP32 hardware and the connection diagram. Next show the Serial Monitor output. Then close Serial Monitor and start the Streamlit dashboard. Demonstrate a normal case and then a risk case.

F.4 Troubleshooting

If the dashboard says ESP32 Not Connected, check that the ESP32 is plugged in and COM5 is correct. Close Arduino Serial Monitor if it is open. If temperature is -127 °C, check the DS18B20 data wire and 4.7 kΩ pull-up resistor. If pH is 4095, check the pH signal connection, grounding and sensor state.

F.5 Images to Add





F.6 Shutdown

After demonstration, stop Streamlit, close the serial connection, disconnect ESP32 power safely and save the final screenshots, source code and report version used for submission.

6.11 Troubleshooting Guide

Problem	Likely Cause	Solution
ESP32 not connected	Wrong COM port or cable	Check Device Manager and select the correct port
Access denied on COM5	Serial Monitor using port	Close Serial Monitor and restart Streamlit
Temperature = -127 °C	DS18B20 not detected	Check 3.3 V, GND, data pin and 4.7k pull-up

pH = 4095 raw	Signal floating/full-scale input	Check PO/GND wiring and calibrate the interface
TDS value unstable	Probe/environment variation	Use stable water sample and calibrate before ppm use
Water value mostly zero	Sensor placement/threshold	Test dry/wet states and select a measured threshold
Streamlit parser empty	Serial labels changed	Restore exact Arduino labels or update parser
Model feature mismatch	Wrong live input columns	Do not feed TDS/water values into old RF model

Troubleshooting should begin with the physical layer and then move upward: power and wiring → Arduino serial output → COM-port access → Python parsing → dashboard logic. This order reduces debugging time because software cannot recover a sensor that is electrically disconnected.

6.7 Summary

The live demonstration confirms the complete software path from ESP32 sensor readings to dashboard status. The prototype is suitable for project demonstration and further development. Production use would require calibrated sensors, robust error handling, data logging, network communication and field validation.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Smart Aquaculture Assistant successfully combines IoT sensing, Python software, a Streamlit dashboard and machine-learning analysis into a final-year project prototype. The ESP32 provides the embedded sensing layer, while the dashboard converts raw serial measurements into understandable status information.The historical aquaculture dataset was used to train a Random Forest Classifier using Temperature, Dissolved Oxygen, pH and Turbidity. The project achieved approximately 99.89% accuracy on the selected test split. At the same time, the live prototype uses Temperature, pH, TDS voltage and water-sensor readings and applies transparent rule-based assessment. Keeping these two paths separate avoids making an incorrect claim that the existing Random Forest model directly consumes the current live sensor inputs.Overall, the project demonstrates a practical foundation for intelligent aquaculture monitoring and provides a clear path toward a more robust system.

7.2 Limitations

- The live prototype uses USB serial communication rather than completed cloud/Wi-Fi data transmission.
- The TDS sensor is displayed as voltage and has not been converted to validated ppm values.
- The HW-038 water sensor is not a calibrated water-depth measurement device.
- The pH interface requires proper multi-point calibration for reliable field use.
- The historical Random Forest model uses DO and turbidity, while the current live prototype uses TDS and water-sensor readings.
- The 99.89% ML result is based on the project test split and requires stronger independent validation.
- No long-term pond trial has been completed.
- Automatic pump/oxygenation control is not included in the current working prototype.

7.3 Future Scope

- Retrain the ML model using the exact calibrated live sensor feature set.
- Add Dissolved Oxygen and turbidity sensors when suitable hardware is available.
- Add Wi-Fi or MQTT communication for remote monitoring.
- Store readings in a database for historical trend analysis.
- Add mobile notifications for abnormal conditions.
- Develop time-series forecasting for future water-quality changes.

- Add automatic aerator or pump control with safety limits.
- Improve sensor calibration and fault detection.
- Validate the system in a real aquaculture pond with expert supervision.
- Use time-based and pond-based evaluation to measure real-world ML generalization.

7.4 Summary

The project provides a complete educational prototype for smart aquaculture monitoring. Its main value is the integration of low-cost sensing, clear visualization and data-driven analysis. Future development should focus on calibration, matching the live ML feature set, field validation and reliable IoT communication.

CHAPTER 8
REFERENCES

[1] R. Baena-Navarro, Y. Carriazo-Regino, and F. Torres-Hoyos, "IoT Monitoring Dataset of Water Quality and Tilapia (Oreochromis niloticus) Health in Aquaculture Ponds in Montería, Colombia," Mendeley Data, 2024, doi: 10.17632/3g2b4sh65m.1.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, M. Duchesnay, and É. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] D. R. Prapti, A. R. Mohamed Shariff, H. Che Man, N. M. Ramli, and T. Perumal, "Internet of Things (IoT)-based aquaculture: An overview of IoT application on water quality monitoring," *Reviews in Aquaculture*, vol. 14, no. 2, pp. 979–992, 2022, doi: 10.1111/raq.12637.

[5] S. Zhao, S. Zhang, J. Liu, H. Wang, J. Zhu, D. Li, and R. Zhao, "Application of machine learning in intelligent fish aquaculture: A review," *Aquaculture*, vol. 540, Art. no. 736724, 2021, doi: 10.1016/j.aquaculture.2021.736724.

[6] D. R. Roy Prapti, A. R. Mohamed Shariff, H. Che Man, N. M. Ramli, and T. Perumal, "Internet of Things-based aquaculture and water quality monitoring," *Reviews in Aquaculture*, 2022.

[7] "Charting the aquaculture internet of things impact: Key applications, challenges, and future trend," *Aquaculture Reports*, vol. 39, Art. no. 102358, 2024, doi: 10.1016/j.aqrep.2024.102358.

[8] M. Flores-Iwasaki, G. A. Guadalupe, M. Pachas-Caycho, S. Chapa-Gonza, R. C. Mori-Zabarburú, and J. C. Guerrero-Abad, "Internet of Things (IoT) Sensors for Water Quality Monitoring in Aquaculture Systems: A Systematic Review and Bibliometric Analysis," *AgriEngineering*, vol. 7, no. 3, Art. no. 78, 2025, doi: 10.3390/agriengineering7030078.

[9] S. M. Roy, M. Masum Beg, S. K. Bhagat, D. Charan, C. M. Pareek, S. Moulick, and T. Kim, "Application of artificial intelligence in aquaculture – Recent developments and prospects," *Aquacultural Engineering*, vol. 111, Art. no. 102570, 2025, doi: 10.1016/j.aquaeng.2025.102570.

[10] "Artificial intelligence in aquaculture: Advancing sustainable fish farming through AI-driven monitoring, optimization, and disease management," *Aquaculture*, vol. 614, Art. no. 743602, 2026, doi: 10.1016/j.aquaculture.2025.743602.

[11] C. Zulkarnain, C. W. Budiyanto, and M. S. Bakar, "A Systematic Literature Review of Dissolved Oxygen and Turbidity Monitoring in Biofloc Aquaculture: IoT and Machine Learning for Water Quality Management," *Journal of Information Systems and Informatics*, vol. 8, no. 3, 2026, doi: 10.63158/journalisi.v8i3.1630.

[12] “Machine Learning on Low-Cost Edge Devices for Real-Time Water Quality Prediction in Tilapia Aquaculture,” *Sensors*, 2025.

[13] F. Aung *et al*., “Artificial intelligence methods used in various aquaculture applications: A systematic literature review,” *Journal of the World Aquaculture Society*, vol. 56, no. 1, Art. no. e13107, 2025, doi: 10.1111/jwas.13107.

[14] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[15] Streamlit, “Streamlit Documentation: A faster way to build and share data apps,” Streamlit Documentation.

[16] Espressif Systems, “ESP32 Series Documentation,” Espressif Systems.

[17] Arduino, “Arduino IDE Documentation,” Arduino Documentation.

[18] Analog Devices / Maxim Integrated, “DS18B20 Programmable Resolution 1-Wire Digital Thermometer.”

[19] Python Software Foundation, “Python Documentation,” Python Documentation.

[20] Joblib Developers, “Joblib Documentation: Persistence Utilities for Python Objects.”

[21] The pandas Development Team, “pandas – Python Data Analysis Library,” pandas Documentation.

[22] Git Project, “Git Documentation,” Git Version Control System.

[23] NumPy Developers, “NumPy Documentation,” NumPy Documentation.

[24] J. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA, USA: O'Reilly Media.

[25] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

CHAPTER 9
APPENDIX

A.1 Project Directory Structure

Path	Description
Smart-Aquaculture-Assistant/	Root project folder
README.md	Project documentation
app.py	Streamlit application
requirements.txt	Python package requirements
ESP32/smart_aquaculture_esp32.ino	ESP32 firmware
ML/fish_health_model.ipynb	ML training notebook
Dataset/aquaculture-dataset.csv.xlsx	Historical dataset
Model/fish_health_model.pkl	Saved Random Forest model
Images/system_architecture.png	System architecture image
Images/dashboard.png	Dashboard screenshot

Table A.1: Project Directory Structure

A.2 Important Commands

Purpose	Command
Install packages	pip install -r requirements.txt
Run Streamlit	streamlit run app.py
Train/save model	Run the ML notebook in Jupyter
Upload code	git add .
Commit	git commit -m "Smart Aquaculture Assistant project"
Push	git push

Table A.2: Important Commands

A.3 Image Insertion Checklist

Figure	What to Insert	Where
Fig. 3.1	Final system architecture diagram	Chapter 3 – System Architecture
Fig. 3.2	Data-flow diagram	Chapter 3 – Data Flow
Fig. 4.2	Clean labelled ESP32 connection diagram	Chapter 4 – ESP32 implementation
Fig. 5.1	Dataset health-status bar chart	Chapter 5 – Dataset results
Fig. 5.2	Jupyter model accuracy screenshot	Chapter 5 – Model results
Fig. 5.3	Final hardware prototype photo	Chapter 5 – Sensor results
Fig. 5.4	GitHub repository screenshot	Chapter 5 – Dashboard/GitHub evidence
Fig. 6.1	ESP32 Serial Monitor screenshot	Chapter 6 – Live monitoring
Fig. 6.2–6.5	Final Streamlit screenshots	Chapter 6 – Demonstration
Image rule	Use a different image for every figure. Never reuse the same photograph, screenshot or diagram.	Entire report

Table A.3: Image Insertion Checklist

CHAPTER 10
SOURCE CODE

10.1 Overview of the Source Code

The source code chapter contains the main software components required to reproduce the prototype: ESP32 firmware, Streamlit dashboard, Random Forest training code and package requirements. The complete project repository should contain these files so that the evaluator can inspect and run the implementation.

10.2 ESP32 Source Code – smart_aquaculture_esp32.ino

```
#include <OneWire.h>
#include <DallasTemperature.h>

#define TEMP_PIN 4
#define WATER_PIN 32
#define TDS_PIN 34
#define PH_PIN 35

OneWire oneWire(TEMP_PIN);
DallasTemperature temperatureSensor(&oneWire);

void setup() {
  Serial.begin(115200);
  temperatureSensor.begin();
  delay(1000);

  Serial.println("================================");
  Serial.println("Smart Aquaculture Assistant");
  Serial.println("ESP32 Sensor Monitoring Started");
  Serial.println("================================");
}

void loop() {
  temperatureSensor.requestTemperatures();
  float temperature = temperatureSensor.getTempCByIndex(0);

  int waterValue = analogRead(WATER_PIN);

  int tdsRaw = analogRead(TDS_PIN);
  float tdsVoltage = tdsRaw * 3.3 / 4095.0;

  int phRaw = analogRead(PH_PIN);
  float pH = 7.0 + (2130 - phRaw) * 0.003;

  Serial.println(" ------------------------ ");

  Serial.print("Temperature: ");
  Serial.print(temperature, 2);
  Serial.println(" C");

  Serial.print("Water Sensor: ");
  Serial.println(waterValue);
```

```
  Serial.print("TDS Voltage: ");
  Serial.print(tdsVoltage, 2);
  Serial.println(" V");

  Serial.print("pH: ");
  Serial.println(pH, 2);

  Serial.println(" ------------------------ ");

  delay(2000);
}
```

10.3 Streamlit Source Code – app.py

```
import streamlit as st
import serial
import time

# ============================================================
# PAGE SETTINGS
# ============================================================

st.set_page_config(
    page_title="Smart Aquaculture Assistant",
    page_icon="🐟🐟",
    layout="wide"
)

st.title("Smart Aquaculture Assistant")
st.subheader("Live ESP32 Sensor Monitoring & Fish Health Prediction")



# ============================================================
# ESP32 CONNECTION
# ============================================================

try:
    ser = serial.Serial(
        "COM5",
        115200,
        timeout=1
    )

    time.sleep(2)

    st.success("✅ ESP32 Connected Successfully — COM5")

except Exception:
    st.error("🔴🔴 ESP32 Not Connected")
    st.warning("Please connect ESP32 and check COM5.")
    st.stop()



# ============================================================
# WATER SENSOR CALIBRATION
```

```
# ============================================================
# Change this value after checking your water sensor readings.
# Example: if water-filled value is above 1000, use 1000.

WATER_NORMAL_THRESHOLD = 1000



# ============================================================
# LIVE DISPLAY AREAS
# ============================================================

st.divider()

st.subheader("📊📊 Live Sensor Values")

col1, col2, col3, col4 = st.columns(4)

temperature_box = col1.empty()
ph_box = col2.empty()
tds_box = col3.empty()
water_box = col4.empty()



st.divider()

st.subheader("🐟🐟 Fish Health Prediction")

prediction_box = st.empty()
prediction_message = st.empty()



st.divider()

st.subheader("📋📋 Sensor Status")

status_col1, status_col2, status_col3, status_col4 = st.columns(4)

temperature_status_box = status_col1.empty()
ph_status_box = status_col2.empty()
tds_status_box = status_col3.empty()
water_status_box = status_col4.empty()



# ============================================================
# SENSOR VARIABLES
# ============================================================

temperature = None
ph_value = None
tds_voltage = None
water_value = None



# ============================================================
```

```
# FUNCTIONS
# ============================================================

def temperature_status(value):

    if 25 <= value <= 30:
        return "Normal"

    elif 23 <= value < 25 or 30 < value <= 32:
        return "At Risk"

    else:
        return "High Risk"


def ph_status(value):

    if 6.5 <= value <= 8.5:
        return "Normal"

    elif 6.0 <= value < 6.5 or 8.5 < value <= 9.0:
        return "At Risk"

    else:
        return "High Risk"


def tds_status(value):

    # This is a prototype voltage range.
    # Proper TDS ppm calibration should be done later.

    if 0.30 <= value <= 1.50:
        return "Normal"

    elif 0.20 <= value < 0.30 or 1.50 < value <= 1.80:
        return "At Risk"

    else:
        return "High Risk"


def water_status(value):

    if value >= WATER_NORMAL_THRESHOLD:
        return "Normal"

    elif value >= WATER_NORMAL_THRESHOLD * 0.5:
        return "Low"

    else:
        return "Very Low"
```

```
# ============================================================
# LIVE ESP32 LOOP
# ============================================================

while True:

    if ser.in_waiting:

        line = ser.readline().decode(
            "utf-8",
            errors="ignore"
        ).strip()


        # ------------------------------------------------------------------
        # TEMPERATURE
        # ------------------------------------------------------------------

        if line.startswith("Temperature:"):

            try:

                temperature = float(
                    line
                    .replace("Temperature:", "")
                    .replace("C", "")
                    .strip()
                )

            except:
                pass


        # ------------------------------------------------------------------
        # WATER SENSOR
        # ------------------------------------------------------------------

        elif line.startswith("Water Sensor:"):

            try:

                water_value = int(
                    line
                    .replace("Water Sensor:", "")
                    .strip()
                )

            except:
                pass


        # ------------------------------------------------------------------
        # TDS
        # ------------------------------------------------------------------
```

```
elif line.startswith("TDS Voltage:"):

    try:

        tds_voltage = float(
            line
            .replace("TDS Voltage:", "")
            .replace("V", "")
            .strip()
        )

    except:
        pass


# ---------------------------------------------------------------
# pH
# ---------------------------------------------------------------

elif line.startswith("pH:"):

    try:

        ph_value = float(
            line
            .replace("pH:", "")
            .strip()
        )

    except:
        pass


# ================================================
# DISPLAY LIVE VALUES
# ================================================

if temperature is not None:

    temperature_box.metric(
        "🌡️ Temperature",
        f"{temperature:.2f} °C"
    )


if ph_value is not None:

    ph_box.metric(
        "🧪 pH",
        f"{ph_value:.2f}"
    )


if tds_voltage is not None:
```

```
    tds_box.metric(
        "🧪 TDS Voltage",
        f"{tds_voltage:.2f} V"
    )


if water_value is not None:

    water_box.metric(
        "💧 Water Level",
        str(water_value)
    )



# ================================================
# CHECK ALL FOUR SENSOR CONDITIONS
# ================================================

if (
    temperature is not None
    and ph_value is not None
    and tds_voltage is not None
    and water_value is not None
):

    temp_result = temperature_status(
        temperature
    )

    ph_result = ph_status(
        ph_value
    )

    tds_result = tds_status(
        tds_voltage
    )

    water_result = water_status(
        water_value
    )



    # ================================================
    # SENSOR STATUS DISPLAY
    # ================================================

    if temp_result == "Normal":

        temperature_status_box.success(
            "🌡️ Normal"
        )

    elif temp_result == "At Risk":

        temperature_status_box.warning(
```

```
            "🌡️ At Risk"
        )

    else:

        temperature_status_box.error(
            "🌡️ High Risk"
        )


    if ph_result == "Normal":

        ph_status_box.success(
            "✅ Normal"
        )

    elif ph_result == "At Risk":

        ph_status_box.warning(
            "⚠️ At Risk"
        )

    else:

        ph_status_box.error(
            "❌ High Risk"
        )


    if tds_result == "Normal":

        tds_status_box.success(
            "✅ Normal"
        )

    elif tds_result == "At Risk":

        tds_status_box.warning(
            "⚠️ At Risk"
        )

    else:

        tds_status_box.error(
            "❌ High Risk"
        )


    # WATER STATUS

    if water_result == "Normal":

        water_status_box.success(
            "💧 Normal"
```

```
    )

elif water_result == "Low":

    water_status_box.warning(
        "💧💧 Low"
    )

else:

    water_status_box.error(
        "💧💧 Very Low"
    )


# ================================================
# FISH HEALTH SCORE
# ================================================

score = 0

if temp_result == "Normal":
    score += 1

if ph_result == "Normal":
    score += 1

if tds_result == "Normal":
    score += 1

if water_result == "Normal":
    score += 1



# ================================================
# FISH HEALTH PREDICTION
# ================================================

if score == 4:

    prediction_box.success(
        "🐟🐟 FISH HEALTH STATUS: STABLE"
    )

    prediction_message.info(
        "✅ All four live sensor conditions "
        "are currently normal."
    )



elif score >= 2:

    prediction_box.warning(
        "⚠️ FISH HEALTH STATUS: AT RISK"
```

```
                )

                prediction_message.warning(
                    "⚠ Some sensor conditions require attention."
                )


            else:

                prediction_box.error(
                    "🚨🚨 FISH HEALTH STATUS: HIGH RISK"
                )

                prediction_message.error(
                    "🚨🚨 Multiple sensor conditions "
                    "require immediate attention."
                )


        # Small delay
        time.sleep(0.1)
```

10.4 Machine Learning Source Code – fish_health_model.ipynb

```
import pandas as pd

df = pd.read_excel("aquaculture-dataset.csv.xlsx")

X = df[[
    "Temperature (°C)",
    "Dissolved Oxygen (mg/L)",
    "pH",
    "Turbidity (NTU)"
]]

y = df["Health Status"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

from sklearn.metrics import accuracy_score
```

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy * 100, "%")

# Sample prediction
new_data = pd.DataFrame([{
    "Temperature (°C)": 27.5,
    "Dissolved Oxygen (mg/L)": 6.3,
    "pH": 8.0,
    "Turbidity (NTU)": 3.3
}])

prediction = model.predict(new_data)

print("Fish Health Status:", prediction[0])

# Save model
import joblib
joblib.dump(model, "fish_health_model.pkl")

print("Model saved successfully.")
```

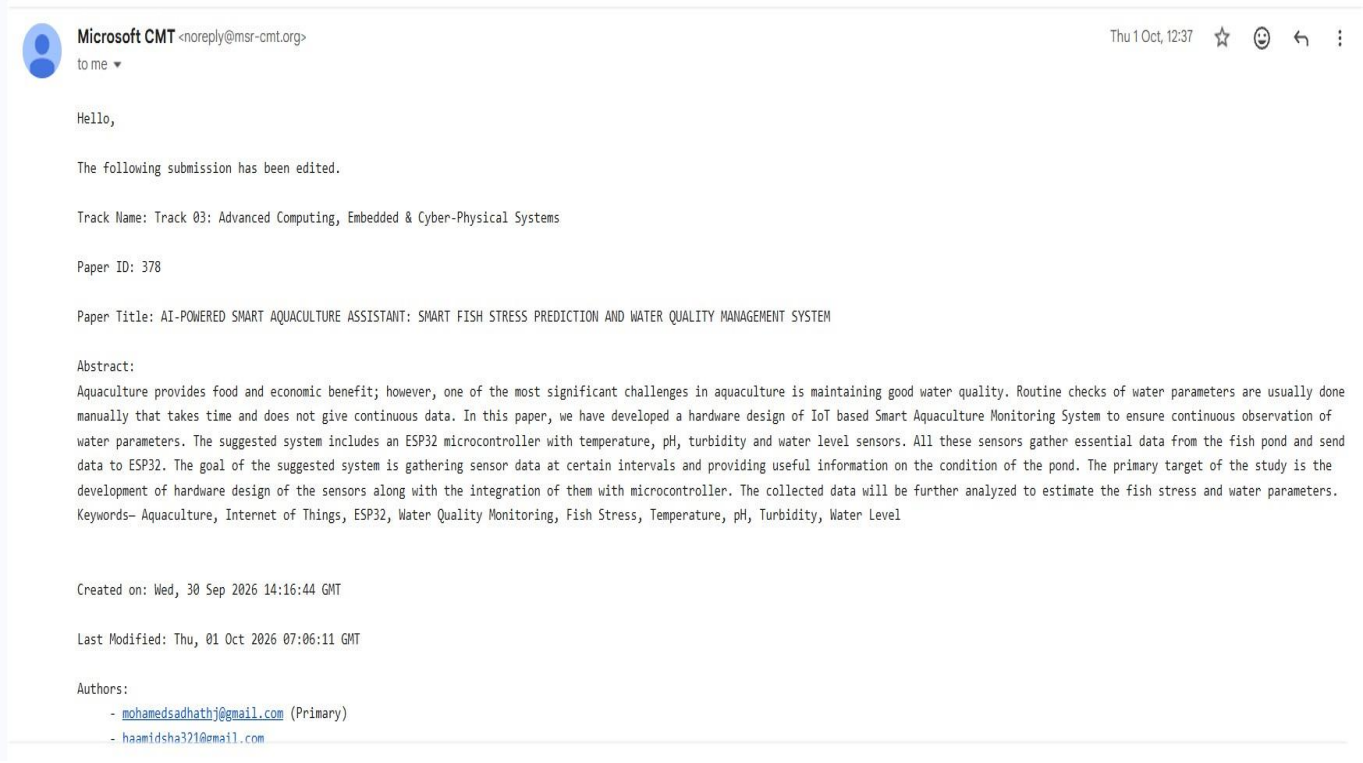
10.5 requirements.txt

```
streamlit
pyserial
pandas
numpy
scikit-learn
joblib
openpyxl
```

10.6 Notes for Running the Source Code

- Upload the ESP32 sketch and select the correct board and COM port in Arduino IDE.
- Close Arduino Serial Monitor before launching Streamlit so that COM5 is free.
- Open the project folder in VS Code or Command Prompt.
- Install the packages listed in requirements.txt.
- Run the dashboard using streamlit run app.py.
- Keep the ESP32 serial labels unchanged unless the Python parser is updated.
- Do not claim calibrated TDS ppm or field-grade pH accuracy without calibration.

CHAPTER 11
CONFERENCE APPLICATION



Microsoft CMT <noreply@msr-cmt.org> Thu 1 Oct, 12:37

to me

Hello,

The following submission has been edited.

Track Name: Track 03: Advanced Computing, Embedded & Cyber-Physical Systems

Paper ID: 378

Paper Title: AI-POWERED SMART AQUACULTURE ASSISTANT: SMART FISH STRESS PREDICTION AND WATER QUALITY MANAGEMENT SYSTEM

Abstract:
Aquaculture provides food and economic benefit; however, one of the most significant challenges in aquaculture is maintaining good water quality. Routine checks of water parameters are usually done manually that takes time and does not give continuous data. In this paper, we have developed a hardware design of IoT based Smart Aquaculture Monitoring System to ensure continuous observation of water parameters. The suggested system includes an ESP32 microcontroller with temperature, pH, turbidity and water level sensors. All these sensors gather essential data from the fish pond and send data to ESP32. The goal of the suggested system is gathering sensor data at certain intervals and providing useful information on the condition of the pond. The primary target of the study is the development of hardware design of the sensors along with the integration of them with microcontroller. The collected data will be further analyzed to estimate the fish stress and water parameters.
Keywords– Aquaculture, Internet of Things, ESP32, Water Quality Monitoring, Fish Stress, Temperature, pH, Turbidity, Water Level

Created on: Wed, 30 Sep 2026 14:16:44 GMT

Last Modified: Thu, 01 Oct 2026 07:06:11 GMT

Authors:
- mohamedsadhathj@gmail.com (Primary)
- haamidsha3210@gmail.com

Conference: iConCISE 2027 (International Conference on Computation, Communication, AND Intelligent Systems in Engineering)

Paper ID: 378

Paper: AI-POWERED SMART AQUACULTURE ASSISTANT: SMART FISH STRESS PREDICTION AND WATER QUALITY MANAGEMENT SYSTEM